TECHNISCHE UNIVERSITÄT DRESDEN

Faculty of Computer Science Institute of Systems Architecture
Chair of Systems Engineering

Master Thesis

Towards automated security evaluation using policies: Concept and case study

Prateek Rohilla

Born on: 4th August 2000 in Bhiwani, India
Matriculation number: 5123833

28th October 2025

First referee
Prof. Dr. Christof Fetzer

Second referee
Dr.-Ing. André Martin

Supervisor
Dipl.-Ing. Julius Wenzel

TECHNISCHE UNIVERSITÄT DRESDEN

Faculty of Computer Science Chair of Systems Engineering

Task for the preparation of a Master Thesis

Course:	Distributed Systems Engineering
Name:	Prateek Rohilla
Matriculation number:	5123833
Matriculation year:	2022
Title:	Towards automated security evaluation using policies: Concept and case study

Objectives of work

The goal of this thesis is to automate the verification of software policies using the Open Policy Agent (OPA) or a comparable framework. In modern software development, ensuring security and compliance has become a question of responsibility and accountability. In critical environments, releases often require approval by designated personnel ("governors") who must sign off on specifications serving as a reference point for later audits.

Manual examination of these specifications can be time-consuming and complex, but much of the verification process follows logical rules that can be formalized and automated. This thesis investigates how such automation can be achieved, including its practical limits. The student will design and implement a proof-of-concept demonstrating automated verification for the *Caddy* web server, which manages TLS and certificate handling.

The use case assumes that governors sign collections of configuration files ("manifests") and that build outputs, test results, and vulnerability scan reports are stored in machine-readable form within one or more data repositories. The student will define verification requirements and formalize rules that check both manifests and collected output data for compliance with security expectations. A preliminary set of requirements includes:

- All code leading to deployment binaries must be properly formatted.
- All code must pass the CI/CD pipeline.
- No known vulnerabilities may exist, except those explicitly documented and approved.

This list is to be extended through appropriate threat analysis methods.

Tasks

- Establish the verification requirements relevant to the governors.
- Design a workflow that collects manifests and output data and provides verification results.
- Define specific rules and policies for the Caddy framework that implement these requirements.
- Implement the designed workflow and policies as a proof-of-concept.
- Perform a threat analysis of the resulting solution.
- Evaluate the performance and verification effort of the proposed workflow.
- Optionally, discuss methods for authenticating verification data using digital signatures.

TECHNISCHE UNIVERSITÄT DRESDEN

Faculty of Computer Science Chair of Systems Engineering

Note: The student grants the Chair of Systems Engineering exclusive ownership and usage rights to any software developed as part of this thesis. These rights extend beyond the completion of the thesis.

First referee:	Prof. Dr. Christof Fetzer
Second referee:	Dr.-Ing. André Martin
Supervisor:	Dipl.-Ing. Julius Wenzel
Issued on:	28th May 2025
Due date for submission:	29th October 2025

Prof. Dr. Christof Fetzer
Supervising professor

Statement of authorship

I hereby certify that I have authored this document entitled *Towards automated security evaluation using policies: Concept and case study* independently and without undue assistance from third parties. No other than the resources and references indicated in this document have been used. I have marked both literal and accordingly adopted quotations as such. There were no additional persons involved in the intellectual preparation of the present document. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.

Dresden, 28th October 2025

Prateek Rohilla

Prateek Rohilla Digitally signed by Prateek Rohilla
Date: 2025.10.28 23:38:57 +01'00'

TECHNISCHE UNIVERSITÄT DRESDEN

Faculty of Computer Science Chair of Systems Engineering

Abstract

Modern DevOps practices accelerate software delivery but come with significant security and governance challenges, often relying on slow and inconsistent manual reviews. This thesis focuses on the automation of release verification using a policy-as-code framework. We introduce the design and implementation of a novel service that systematically collects evidence from build pipelines, code hosting platforms, and vulnerability databases. This evidence is combined into a canonical JSON report for further analysis and policy enforcement.

A key contribution is a custom Go plugin for the Open Policy Agent (OPA), which serves as a proof of concept to extend the policy engine beyond Rego in another language. This allows for the policy engine to be more flexible and extensible. Our evaluation proved the effectiveness and scalability of the system. It successfully enforced policies in a GitHub Actions workflow and demonstrated manageable performance overhead, scaling from few seconds for small projects to few minutes for really large codebases. Our analysis also revealed critical insights and limitations. Multiple threats and mitigations were also identified and discussed. We conclude that our architecture provides a robust model and a powerful tool to assist human governance in the software delivery process.

Contents

Abstract . . . VII

List of Figures . . . XI

List of Tables . . . XIII

Symbols and Acronyms . . . XVII

1 Introduction . . . 1
- 1.1 Motivation . . . 1
- 1.2 Goal . . . 2
- 1.3 Scope of the Thesis . . . 3
- 1.4 Thesis Structure . . . 3

2 Background . . . 5
- 2.1 Continuous Integration and Continuous Delivery . . . 5
  - 2.1.1 Role in This Work . . . 6
- 2.2 Software Provenance . . . 6
  - 2.2.1 Provenance Collection . . . 6
  - 2.2.2 Provenance Representation . . . 6
  - 2.2.3 Frameworks for Supply Chain Integrity . . . 7
  - 2.2.4 Provenance Utilization . . . 8
- 2.3 Open Policy Agent . . . 8
  - 2.3.1 Implications for the Prototype . . . 9
  - 2.3.2 Source code management and protection . . . 9
  - 2.3.3 Runner Isolation . . . 10
  - 2.3.4 Confidential Computing for Signing and Provenance . . . 10
  - 2.3.5 Summary . . . 10

3 Related Work . . . 11
- 3.1 Evidence Generation and Tool Aggregation . . . 11
  - 3.1.1 Mixed-tool orchestration and unified reporting . . . 11
  - 3.1.2 Supply chain “smell” detection . . . 12
  - 3.1.3 Software Bill of Materials (SBOM) tools and landscape studies . . . 12

3.2 Provenance, Integrity, and Attestation Frameworks . . . 12
3.2.1 in-toto and SLSA . . . 12
3.2.2 Cryptographic canonicalization and report integrity . . . 12
3.3 Policy as Code and Enforcement . . . 13
3.3.1 Policy management frameworks . . . 13
3.4 Threat Modeling and Pipeline Security . . . 13
3.4.1 Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege (STRIDE)-based analysis . . . 13
3.4.2 Government guidance and standards . . . 14
3.4.3 Pipeline hardening and zero trust . . . 14
3.5 Interoperability, Standards, and Formats . . . 14
3.5.1 SBOM and Software Package Data Exchange (SPDX) . . . 14
3.5.2 Interoperability trade-offs . . . 14
3.6 Synthesis . . . 14

4 Methods and Implementation . . . 17
4.1 Method . . . 17
4.1.1 Goals . . . 17
4.1.2 Problem Analysis . . . 18
4.1.3 Search for the right solution . . . 18
4.2 Verification service: Design and Implementation . . . 19
4.2.1 Coordination Layer . . . 20
4.2.2 Report Construction . . . 21
4.2.3 Static Analysis and Linting . . . 22
4.2.4 Formatting Verification . . . 22
4.2.5 Vulnerability Scanning . . . 22
4.2.6 Commit Authorship Verification . . . 22
4.2.7 Manifest File Checks: Open Ports and Environment Variables . . . 23
4.2.8 Provenance Verification . . . 23
4.2.9 Manifest Scanning . . . 24
4.2.10 Aggregation Logic and Report generation . . . 24
4.2.11 Continuous integration context . . . 24
4.3 Open Policy Agent Integration . . . 25
4.3.1 Process Flow . . . 25
4.4 Custom OPA Plugin . . . 25
4.4.1 JSON Report Structure . . . 26
4.4.2 Policy Evaluation Layer . . . 27
4.5 Continuous Integration Automation . . . 27
4.5.1 Method Outline . . . 28
4.5.2 Phase 1: Build and Provenance Generation . . . 28
4.5.3 Phase 2: Verification and Policy Evaluation . . . 28
4.5.4 Gate . . . 28
4.6 Summary . . . 28

5 Results . . . 31
5.1 Evaluation and Results . . . 31
5.1.1 Evaluation Setup . . . 31

5.1.2 Quantitative Results: Execution Time on Local Hardware 31
5.1.3 Qualitative Results 36
5.2 Policy-as-Code Evaluation Demonstration on GitHub 36
5.2.1 Overview 37
5.2.2 GitHub Configuration 37
5.2.3 Workflow Structure 37
5.2.4 Policy Enforcement Scenarios 38
5.2.5 Discussion 41
5.3 Security Analysis: A Threat Model of the Verifier Service 41
5.3.1 Introduction and Methodology 41
5.3.2 Analysis of Threats 42
5.4 Limitations and Critical Analysis 43

6 Conclusion and Further Work 47
6.1 Conclusion 47
6.2 Outlook and Further Work 48
6.2.1 Expanding Ecosystem and Tooling Support 48
6.2.2 Deepening Policy Enforcement and Verification Logic 48
6.2.3 Bridging Pre-Deployment and Runtime Security 49
6.2.4 Addressing Implementation Challenges 49
6.2.5 Solving the Determinism Problem in the Verifier Service 49
6.2.6 Alternative Approaches for Commit Signature Verification 50
6.2.7 Optimizations for Better Scalability 50
6.2.8 The Indispensable Role of Human Oversight 51
6.2.9 Final Remarks 51

Bibliography 51

A Appendix 59
A.1 Example JSON report schema with measurement results and attestation data 59
A.2 OPA policy file 62
A.3 OPA plugin output 64
A.4 OPA evaluation failure due to a port not allowed 64

List of Figures

2.1 The relationship between continuous integration, delivery, and deployment[4]. 6
2.2 Open Policy Agent Setup Diagram 9

4.1 High level design of verification service 21
4.2 Commit signature verification against allowed keys 23
4.3 Policy Evaluation based on custom rego policy and OPA plugins 27
4.4 Continuous Integration (CI) Pipeline – Phase 1: Build and Provenance Generation 30
4.5 CI Pipeline – Phase 2: Verification and Policy Evaluation 30

5.1 Runtime vs. Lines of Code (Log-Log Scale) 33
5.2 Runtime vs. Number of Commits (Log-Log Scale) 33
5.3 Average duration per verification step (Caddy repository, 10 runs) 34
5.4 Total runtime per report (Caddy repository, 10 runs) 34
5.5 Repository structure showing Go source files, `.github/workflows/`, and `policy.rego`. 38
5.6 Branch protection rule configuration for the main branch with required approvals and verification checks. 39
5.7 Status check configuration enforcing the `verify_and_validate` workflow as mandatory before merging. 39
5.8 Pull request with failing `verify_and_validate` check due to unverified commit. The merge option is disabled for the developer. 40
5.9 Successful run of the verification and policy evaluation workflow allowing the merge. 41

List of Tables

5.1 Execution time across repositories (local machine, 5 runs each) 32
5.2 Execution time on GitHub Actions (5 runs each) 36
5.3 Summary of identified threats and mitigations (STRIDE framework). 43

Symbols and Acronyms

CD Continuous Delivery / Continuous Deployment

DAG Directed Acyclic Graphs

CI Continuous Integration

OCI Open Container Initiative

CISA Cybersecurity and Infrastructure Security Agency

CLI Command Line Interface

CVSS Common Vulnerability Scoring System

GPG GNU Privacy Guard

JSON JavaScript Object Notation

LOC Lines of Code

OIDC OpenID Connect

OPA Open Policy Agent

OSV Open Source Vulnerabilities

PROV Provenance (W3C Provenance Data Model)

RAM Random Access Memory

W3C World Wide Web Consortium

RFC Request for Comments

SARIF Static Analysis Results Interchange Format

SAST Static Application Security Testing

SBOM Software Bill of Materials

SCA Software Composition Analysis

SDLC Software Development Life Cycle

SHA Secure Hash Algorithm

SLSA Supply-chain Levels for Software Artifacts

SPDX Software Package Data Exchange

STRIDE Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege

UML Unified Modeling Language

VM Virtual Machine

YAML YAML Ain't Markup Language

1 Introduction

The adoption of DevOps and CI, Continuous Delivery / Continuous Deployment (CD) practices has dramatically increased the speed and agility of software delivery. While this accelerates innovation, it has introduced new and complex challenges for security and governance.

In modern organizations, software changes must comply with internal policies and external regulations such as the EU Cyber Resilience Act [47], the NIS2 Directive [27], or sector-specific standards like ISO/IEC 27001 [18] and the BSI IT-Grundschutz [23]. Governors are responsible for approving such changes by verifying that technical and procedural requirements are satisfied before release. As development environments expand, the number and complexity of required checks have surpassed what can be reliably handled through manual inspection. Effective governance thus requires automation and reliable tooling to help in verifying compliance, correctness and trustworthiness of all such artifacts.

1.1 Motivation

Modern software supply chains have evolved into deep interconnected ecosystems of dependencies, components, and build metadata [61]. As noted by NIST and the OpenSSF [59, 74], such complexity exceeds the capacity of manual review, motivating the need for automation and policy-based verification frameworks [11, 66].

The increasing interconnection directly affects how software is built and delivered. Modern releases now consist of numerous components, dependencies, and metadata accumulated across multiple phases of the Software Development Life Cycle (SDLC). The widespread adoption of microservice architectures, containerized deployments, and continuous rollout strategies have further increased the number of moving parts that have to be tracked and checked [61, 59, 74].

However, ensuring that such releases will continue to be secure and compliant relies still in large part on traditional governance mechanisms. Conventional methods such as Segregation of Duties and the “four-eyes” principle [28], though effective in theory, continue to dominate practice and impose significant overhead [3, 39]. These methods are adding repetitive manual reviews leading to reviewer fatigue, and hence an increased likelihood of human error and lesser assurance [15].

Recent supply-chain attacks such as SolarWinds [32] and Codecov [12] have shown that the most common causes of security vulnerabilities are not only susceptible code or third-party dependencies but also vulnerabilities in build pipelines, provenance verification, and manual supervision in the release processes [57, 38, 75]. These incidents underline the urgent need for verifiable and automated assurance mechanisms.

In response, emerging DevSecOps practices—such as automated policy enforcement, attestation, and provenance tracking—aim to introduce consistency and traceability into release governance. These developments demand an industry-wide, standards-based method of exchanging the verification evidence throughout the entire SDLC. Existing solutions, however, often address isolated concerns (e.g., binary integrity or dependency management) rather than providing an integrated approach that spans pipeline trust, provenance tracking, and vulnerability oversight [66, 58, 70].

Such fragmentation exposes organisations to compliance risks, delayed detection of security issues, and slower deployment cycles. To mitigate these problems, several engineering organizations have adopted automated security assessment frameworks [11, 77]. These systems replace error-prone and informal approval steps with auditable, policy-driven controls. For example, Policy-as-Code [52] shifts approvals from discretionary human decisions to machine-verifiable rules. This transition strengthens human oversight by removing monotonous tasks and allowing engineers to focus on higher-level risk assessment and design considerations. Embedding such controls directly into release pipelines establishes delivery lifecycles that are secure by design, resilient, and inherently auditable.

Building upon this motivation, the next section defines the concrete objectives and scope of the work, outlining how these challenges are addressed.

1.2 Goal

The primary goal of this thesis is to investigate, design, and validate an automated system for software release governance using a Policy-as-Code [50] approach. The objective is to develop a Proof-of-Concept (PoC) targeting specific Golang based open source projects (e.g. Caddy, kubernetes, etc.), that demonstrates how the manual, error-prone task of release verification can be automated.

To achieve this goal, the following key objectives are defined:

Architecture: Design an architecture for a "Verifier Service" capable of aggregating and normalizing diverse metadata, provenance, and dependency information associated with a software release into a unified, queryable format.

Policy Framework: Formalize a representative set of security requirements into custom policies using a declarative policy language. This includes checks for code quality, CI/CD pipeline integrity, and vulnerability management, with documented exceptions.

Proof-of-Concept: Implement a PoC of this Verifier Service, leveraging the Open Policy Agent (OPA) framework to evaluate aggregated release data against the defined policies.

Evaluation: Assess the feasibility and performance of the solution by integrating it into a CI pipeline, measuring its effect on build times and its potential to reduce manual review effort.

Scope of this work is focused on the verification of release artifacts and their associated metadata. The mechanisms for cryptographically signing the final attestation and the secure distribution of artifacts are considered out of scope for this thesis.

1.3 Scope of the Thesis

An automation framework is presented that can be used to evaluate software releases for security and compliance requirements. At its core lies a *Verifier Service* that consolidates Supply Chain Metadata data into a standardized schema, enabling coherent and cross-cutting assessment of release integrity[43]. Security specifications are defined as policies using the Open Policy Agent (OPA) and its Rego language, allowing configurable, auditable verification consistent with known compliance and governance standards[36, 53].

While existing approaches often focus on SBOMs, this work extends verification to include code quality, CI/CD pipeline integrity, and vulnerability management [60]. Hence, the emphasis remains on release-level assurance; upstream development practices and post-deployment observability are excluded.

Verification framework is validated through proof-of-concept integration in a CI/CD pipeline. Evaluation metrics are used to examine the feasibility, runtime overhead, and effectiveness in enforcing key security properties.

1.4 Thesis Structure

The work is organized into six chapters that address the research objectives systematically.

Chapter 2 – Background introduces core concepts in software supply chain security, policy-as-code frameworks, and the Open Policy Agent (OPA) ecosystem.

Chapter 3 – Related Work surveys existing efforts toward automated software release verification and identifies key research gaps.

Chapter 4 – Implementation describes the design and development of the Verifier Service, its architecture, policy model, and CI/CD integration.

Chapter 5 – Evaluation and Results evaluates the implementation through performance metrics, security coverage analysis, and integration testing.

Chapter 6 – Conclusion and Future Work summarizes findings, discusses current limitations, and suggests future improvements.

Each chapter builds upon the previous to form a cohesive exploration of automated release verification through policy-as-code methods.

2 Background

This chapter introduces the fundamental concepts, technologies, and frameworks underlying this thesis, defining terminology and methodological principles for the prototype implementation and evaluation. In particular, the chapter addresses Continuous Integration and Continuous Delivery (CI/CD), software provenance, and policy enforcement using the Open Policy Agent (OPA). Furthermore, complementary concepts such as segregation of duties, reproducibility, and confidentiality mechanisms are discussed, since they play a critical role in securing modern software supply chains.

2.1 Continuous Integration and Continuous Delivery

Modern software engineering relies on automation to transform a source code repository into a deployable software artifact. (CI) and (CD) [4] are central to this paradigm (see Figure 2.1). They improve system reliability and shorten feedback cycles by automating builds, tests, and deployments. In this thesis, they serve as the technical backbone for enforcing quality, security, and compliance checks.

Continuous Integration (CI) is a practice in which developers frequently integrate code changes into a shared version-controlled repository. Each integration triggers an automated build and test sequence typically involving compilation, unit testing, and static analysis. Its primary purpose is to identify integration errors at the earliest possible stage, thereby reducing the cost of defect resolution and maintaining a stable state of the main development branch.

Continuous Delivery (CD) extends CI by automating the preparation of software for release. Beyond compilation and testing, CD includes packaging, deployment into staging environments, execution of acceptance tests, and security scanning. The overarching objective is to ensure that software artifacts can be deployed into production environments at any time, with minimal human intervention and reduced operational risk.

Pipelines implement CI and CD as structured automated workflows. A pipeline orchestrates ordered stages such as dependency resolution, static and dynamic analysis, integration testing, artifact packaging, and deployment. Its declarative nature reduces reliance on manual configuration, thereby limiting the risk of human error.

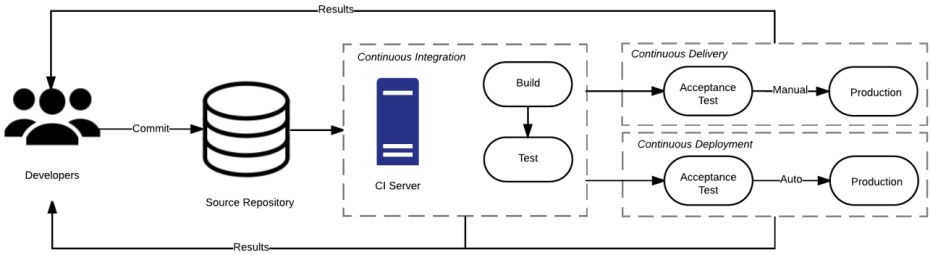

Figure 2.1: The relationship between continuous integration, delivery, and deployment[4].

2.1.1 Role in This Work

Here, a CI/CD pipeline is employed to glue together the various stages of the verification and evaluation process. Every new commit triggers automated workflows that apply these checks in a deterministic manner. It also allows for the governor to read the results of the verification and evaluation process and decide if the build is allowed to proceed to the next stage. This reduces reliance on checks done by a human governor, reducing the risk of human error and ensures consistent application of quality and security standards across all contributions.

The automation of builds and deployments produces metadata that forms the foundation of software provenance.

2.2 Software Provenance

Software provenance [5] refers to the comprehensive record of a software artifact's history, origin, and transformation. It documents the artifact's entire lifecycle, including authorship, build environment, dependencies, and deployment conditions. Such provenance data underpins traceability, reproducibility, and integrity verification, forming a foundation for secure and transparent software supply chains. The concept can be understood across three main stages: *collection*, *representation*, and *utilization*.

2.2.1 Provenance Collection

The first stage, provenance collection, concerns the systematic extraction and recording of metadata produced during software development and build activities. This information may be gathered through static or dynamic analyses, or captured automatically by existing development infrastructure. Version control systems, build automation frameworks, and CI/CD pipelines are primary data sources, as they inherently generate valuable contextual information. For instance, each commit in a version control repository captures authorship and timestamps, while advanced build systems can log compilation commands, dependency graphs, and output binaries. Together, these data points enable later verification of reproducibility and authenticity, linking code to its build process in a verifiable manner [41, 1].

2.2.2 Provenance Representation

Once collected, provenance data must be structured into a standardized and interpretable form. This representation ensures that diverse tools and systems can exchange and validate provenance consistently. A key representation mechanism in this context is the ***(!** ((!)SBOM)*.

Software Bill of Materials (SBOM)

A *Software Bill of Materials* (SBOM) is an inventory that enumerates all components, libraries, modules, binaries, and dependencies used to build a software artifact. It provides a transparent view of the artifact's composition, thereby supporting provenance tracking, vulnerability management, and license compliance. In effect, an SBOM bridges raw provenance data and actionable security insights.

To promote interoperability, several standardized SBOM formats have been developed. Among the most widely adopted are SPDX and CycloneDX. SPDX (Software Package Data Exchange) [22] models software components and their interrelations with emphasis on licensing and compliance metadata, while CycloneDX [46] targets security-oriented use cases such as vulnerability identification and dependency analysis. Although these formats are not directly employed in the implementation of this thesis, they exemplify how structured metadata can facilitate automated verification and exchange of provenance information across supply-chain systems.

Graph-based Provenance Models

Beyond SBOMs, provenance models represent causal and temporal relationships between artifacts, processes, and agents. Directed Acyclic Graphs (DAG)s are commonly used, where nodes denote entities (files, binaries, processes, agents) and edges encode relationships such as `used`, `wasGeneratedBy`, or `wasDerivedFrom`.

W3C PROV World Wide Web Consortium (W3C) PROV [1] is a data representation model and a general framework for expressing provenance information in a structured and interoperable form. The standard, established by the W3C, defines three main classes [2]:

`Entity` a physical, digital, or conceptual item
`Activity` a process that uses or generates entities
`Agent` a person, organization, or system responsible for an activity

Relationships such as `wasGeneratedBy`, `used`, and `wasDerivedFrom` describe how artifacts evolve over time. The model supports extension and multiple serializations (e.g., PROV-XML, PROV-N, PROV-O under OWL/RDF), enabling interoperability between heterogeneous systems.

In software supply chains, SBOM components can map to Provenance (W3C Provenance Data Model) (PROV) *entities*, build steps to PROV *activities*, and developers or build services to PROV *agents*. Extensions of PROV (e.g., ProvONE) add domain-specific structure for capturing workflow plans or execution traces.

2.2.3 Frameworks for Supply Chain Integrity

Provenance becomes most valuable when it is enforced and verified automatically to maintain supply-chain integrity. Frameworks such as *in-toto* and *SLSA* build on provenance concepts to provide verifiable guarantees about how software artifacts are produced.

in-toto [7] defines cryptographically verifiable supply chains. A layout specifies authorized steps and actors, and each actor generates signed link metadata after completing a step.

Final verification ensures that only the correctly signed links match the declared layout, preventing unauthorized modification or omission of steps.

Supply-chain Levels for Software Artifacts (SLSA) (Supply-chain Levels for Software Artifacts) [42] introduces a tiered assurance model ranging from basic build provenance (SLSA 1) to fully hermetic, reproducible builds on hardened infrastructure (SLSA 4). SLSA prescribes criteria for securing build environments and artifact provenance, emphasizing reproducibility and cryptographic verification across the build and release pipeline.

Comparison and Rationale for Framework Selection

Other provenance approaches, such as SPDX and CycloneDX, focus mainly on representing component metadata and relationships. In contrast, SLSA and in-toto address the integrity of the *process* that produces software artifacts. SLSA defines measurable levels of build assurance, while in-toto enforces step-by-step attestation of actions within the supply chain. Such a process-oriented perspective aligns closely with modern CI/CD practices.

For this thesis, SLSA and in-toto were selected because they provide practical, well-supported frameworks for verifiable software builds in Golang-based projects. Both integrate seamlessly with GitHub Actions and Open Container Initiative (OCI) registries, offer native tooling for provenance attestation, and are supported by an active open-source community [71]. Their combined properties enable the construction of a transparent, reproducible, and secure build pipeline.

2.2.4 Provenance Utilization

Collected and represented provenance can serve several purposes:

Security: Detect unauthorized changes and mitigate supply-chain attacks. Provenance enables quick identification of affected components during vulnerability disclosures.

Debugging: Trace defects to their origin by analyzing the historical chain of transformations that produced an artifact.

Reproducibility, Compliance, and Forensics: Reproduce builds, demonstrate compliance in regulated environments, and perform post-incident analysis of artifact creation.

Reproducible builds exemplify these principles by producing bit-identical binaries from identical inputs. Such metadata underpins higher-level assurance frameworks such as SLSA and in-toto, forming the foundation for trusted software supply chains.

2.3 Open Policy Agent

Open Policy Agent (OPA) serves as a general-purpose policy engine that separates decision logic from application code. Services query OPA with structured input (JSON), which is evaluated against policies written in the Rego language (see Figure 2.2). Results, such as "allow" or "deny," are returned in JSON format. Decoupling decision logic enables centralized policy management and consistent enforcement across heterogeneous systems.

Rego is a declarative, logic-based language for defining policies over structured data. Rules derive conclusions from input facts and external sources, allowing precise control and

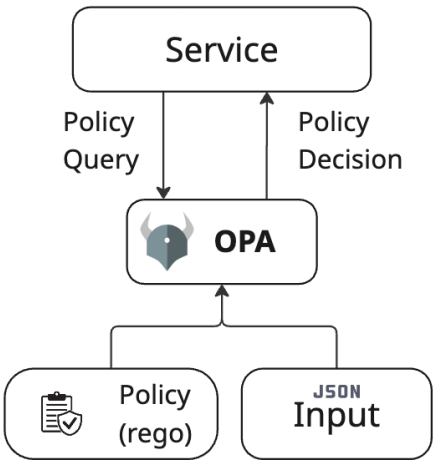

Figure 2.2: Open Policy Agent Setup Diagram

transparent auditing. Together, OPA and Rego realize the policy-as-code model, aligning compliance and security enforcement with operational environments. Policies defined in OPA complement cryptographic verification by ensuring organizational constraints are met before deployment.

2.3.1 Implications for the Prototype

The core objective of this work was to investigate how OPA-based policy verification can be applied to a realistic, production-oriented system. To demonstrate this, the developed prototype integrates OPA and Rego concepts as enforceable controls within the GitHub-based workflow of the `Caddy` project, which serves as a representative target system. This practical integration illustrates how policy-as-code principles can be operationalized in an existing software delivery pipeline, enforcing both organizational and cryptographic requirements. Subsequent subsections describe the specific enforcement mechanisms applied and their impact on controlled repository and deployment behavior.

2.3.2 Source code management and protection

Branch protection rules ensure that release-critical branches cannot be modified without proper validation. In the context of the prototype, these rules emulate organizational policies which require platform specific setup (Github in this case).

Required Reviews: At least one independent reviewer must approve a pull request before merging into the release branch. This enforces the four-eyes principle and reduces the risk of unreviewed or malicious code changes.

Verified Commits Only: All commits merged into protected branches must be signed using a digital signature. A thourough check is performed in a later stage. This ensures that only authorized authors contribute to release-critical code history.

Protected Branches: The main and release branches are configured as protected. A certain set of rules are applied to the branches, which we discuss in the Chapter 4.

Policy Reflection: The OPA policy layer validates the corresponding evidence ensuring that all granular checks are in line with the policy. Any violation is strictly enforced and logged for the governor to review.

This approach demonstrates how repository-level access controls and cryptographic verification can be expressed as enforceable, machine-verifiable policies within the OPA-based compliance framework.

2.3.3 Runner Isolation

Jobs execute on GitHub-hosted runners by default. Self-hosted runners are usually used for hardware features that are required (e.g., confidential computing). These runners are isolated by:

Network Policies: Egress rules restrict outbound connections, reducing data exfiltration risk.

Ephemeral State: Runners reset working directories and credentials between jobs to prevent persistence.

Trust Segmentation: Separate runners are designated for staging and production, preventing lateral movement across environments.

2.3.4 Confidential Computing for Signing and Provenance

Artifact signing and provenance generation can be executed within trusted execution environments:

Remote Attestation: The enclave or Virtual Machine (VM) generates attestation evidence describing its code and configuration. The release controller verifies this evidence before accepting outputs.

Sealed Key Storage: Cryptographic signing keys are sealed inside the enclave or VM and cannot be extracted even by privileged system administrators.

Provenance Embedding: SLSA-compliant provenance is generated in the same environment, ensuring that the attested build and signing process is cryptographically tied to the final artifact.

2.3.5 Summary

By combining GitHub's built-in protections (branch rules, environment approvals, OpenID Connect (OIDC)) with external enforcement mechanisms (confidential computing, provenance verification), the prototype establishes a hardened CI/CD pipeline. It transforms human policies into enforceable, auditable, and cryptographically verifiable guarantees, aligning the pipeline with state-of-the-art practices for software supply chain security.

3 Related Work

This chapter surveys prior research and industrial efforts that address verifiable, policy-driven security in software supply chains. The works are grouped into five themes corresponding to the aspects relevant for this thesis:

1. Evidence generation and tool aggregation
2. Provenance, integrity, and attestation frameworks
3. Policy as code and enforcement layers
4. Threat modeling and pipeline security
5. Interoperability, standards, and report formats

For each group, the discussion highlights the addressed problem, the proposed solutions, the identified benefits and limitations, and the resulting implications for the design space.

3.1 Evidence Generation and Tool Aggregation

This section reviews existing methods for combining and managing security evidence across tools. It outlines how security scanners and analyzers are coordinated through orchestration frameworks that unify their outputs. The discussion also contrasts current practices in tool integration, heuristic detection, and SBOM analysis to set the stage for later sections on provenance and policy enforcement.

3.1.1 Mixed-tool orchestration and unified reporting

Security pipelines often integrate multiple tools such as Static Application Security Testing (SAST), Software Composition Analysis (SCA), and secret scanners, creating challenges of heterogeneity and duplication [44]. Prior efforts introduced orchestration layers and dashboards that collect and normalize results.

Platforms such as DefectDojo [76] provide orchestration and unified dashboards [56] and commercial vulnerability management systems import outputs from scanners, map them to a unified model, deduplicate issues, and provide consolidated dashboards for analysts. These solutions improve visibility and centralize security reporting but generally operate after scans complete and are not part of the enforcement chain in CI/CD. Thus, while aggregation improves situational awareness, it does not yet enforce or verify compliance at runtime.

The Static Analysis Results Interchange Format (Static Analysis Results Interchange Format (SARIF)) [33] was proposed to standardize static analysis results. It has been adopted by

platforms such as GitHub Code Scanning [49, 62]. SARIF reduces the need for custom parsers, but its focus remains on static analysis results. Other domains such as provenance, environmental verification, or policy aggregation are outside its scope, indicating the need for broader canonical reporting structures.

3.1.2 Supply chain "smell" detection

Recent research explores heuristic detection of software supply chain "smell" (anti-patterns) that indicate potential risks in open-source ecosystems. For instance, *Dirty-Waters* [54] analyzes JavaScript projects and flags issues such as unpinned dependencies, outdated or deprecated libraries, and missing release tags. These indicators help identify systemic weaknesses in dependency management but stop short of providing verifiable attestations or enforceable security policies. Their primary aim is awareness and early detection rather than cryptographic assurance. Similar heuristic-based detection ideas also appear in broader studies of security smells in infrastructure code [21] and surveys of supply chain risk analysis [35, 80]. Together, these approaches demonstrate the growing importance of automated inspection but also reveal a lack of integration between detection and formal verification layers.

3.1.3 SBOM tools and landscape studies

A large-scale survey compared more than 80 SBOM generation and consumption tools, analyzing differences in supported formats, integration capabilities, and downstream usability [55]. While these tools improve transparency in dependency usage, they remain limited to descriptive reporting. This observation connects directly to the next group of works that focus on provenance and integrity verification, which attempt to extend reporting into cryptographically verifiable evidence.

While evidence aggregation provides visibility, it does not yet guarantee trust. This motivates work on provenance and integrity mechanisms.

3.2 Provenance, Integrity, and Attestation Frameworks

The following works address how to generate and verify trusted metadata across build and release pipelines. They establish the foundation for verifiable evidence, which later enables policy enforcement mechanisms.

3.2.1 in-toto and SLSA

Both in-toto [7] and SLSA [78] define structured approaches for supply-chain attestation. in-toto provides cryptographically signed link metadata for each build step, while SLSA extends this idea into progressive assurance levels for provenance and reproducibility. These frameworks form the conceptual foundation for the verification model used in this thesis but differ in scope: in-toto focuses on artifact integrity, whereas SLSA standardizes the broader assurance lifecycle.

3.2.2 Cryptographic canonicalization and report integrity

Attestation frameworks rely on deterministic encoding to ensure that identical logical content always produces the same cryptographic hash or signature. For example, the *Dead Simple Signing Envelope* (DSSE) and the in-toto attestation format both depend on canonical JSON serialization to guarantee verifiable signatures across toolchains [64].

Because standard JSON lacks a defined key order, canonicalization schemes such as Request for Comments (RFC) 8785 [8] enforce strict serialization rules—sorted keys, normalized numbers, and consistent whitespace handling. This enables reproducible hashing and signature validation for semantically identical documents. Although prior work has explored composite attestations and chained verification, the architectural boundary between evidence generation and policy enforcement remains underexplored [65].

Having established how provenance data can be verified, the next question is how to act upon it through enforceable rules.

3.3 Policy as Code and Enforcement

Policy as Code expresses security and compliance rules in machine-readable form so they can be automatically checked and enforced by systems instead of manually reviewed by humans. It provides a bridge between evidence and action.

OPA is widely used for policy-as-code enforcement in contexts such as Kubernetes admission control and infrastructure-as-code validation [37, 17]. OPA has also been proposed for use in CI/CD pipelines to reject unsafe manifests or enforce organizational rules [20, 63]. Recent work extends these ideas to supply-chain security and build-time attestation checks [41, 31].

3.3.1 Policy management frameworks

Several vendors extend OPA or similar technologies into centralized platforms (e.g., Harness Policy as Code) [19]. These frameworks support global policy management, severity assignment, and enforcement across environments. Their primary focus is on deployment and governance layers. Fewer works address enforcement based on intermediate security evidence or composite verification reports within the pipeline itself. This distinction points to a gap between policy definition and evidence-driven enforcement, which is further reflected in the body of research focusing on pipeline security threats.

Enforcement alone is insufficient without understanding the underlying threat landscape these policies aim to mitigate.

3.4 Threat Modeling and Pipeline Security

This group of works addresses the security posture of CI/CD systems and their threat landscape. It complements policy frameworks by identifying the risks those policies should mitigate.

3.4.1 STRIDE-based analysis

The STRIDE model is a structured approach to identifying and classifying security threats. It defines six categories-spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege where each category corresponds to a specific type of adversarial action.

STRIDE-based methods have been applied to CI/CD pipelines, mapping common threats (tampering, spoofing, repudiation, information disclosure, denial of service, and elevation of privilege) to recommended controls. Such analyses align attack scenarios with frameworks like SLSA, clarifying where mitigations are required. These works emphasize threat categorization but generally stop short of proposing integrated enforcement mechanisms [16].

They help define the adversarial model but not the operational verification required to counter it.

3.4.2 Government guidance and standards

Documents such as NIST SP 800-204D and the Cybersecurity and Infrastructure Security Agency (CISA) Secure Software Development Attestation Form provide prescriptive recommendations for software supply chain security [24, 45]. Common elements include mandatory artifact signing, verifiable provenance, isolated build environments, and auditability. These documents set expectations for industry adoption and compliance but leave room for technical realization [29]. They serve as policy anchors that highlight the need for verifiable and automatable implementations.

3.4.3 Pipeline hardening and zero trust

Industry publications such as Google's BeyondProd whitepaper emphasize applying zero-trust principles within CI/CD environments, strict identity controls, and least-privilege roles for pipeline agents [26, 10, 30]. These works focus on operational practices rather than technical evidence aggregation or policy engines. They contextualize enforcement strategies but do not define the necessary data structures for automated reasoning, leading naturally into a discussion on interoperability and standardized reporting.

3.5 Interoperability, Standards, and Formats

This section examines how standard formats enable or constrain the exchange of verifiable evidence between tools and organizations [13, 34]. It provides the foundation for reusable and interoperable security attestations.

3.5.1 SBOM and SPDX

The Software Package Data Exchange (SPDX) format provides a standardized schema for SBOMs, including component, license, and relationship information. It has been standardized as ISO/IEC 5962:2021 [13]. SPDX is widely used in SBOM ecosystems and facilitates interoperability across tooling [14]. By offering a common vocabulary, it allows independent systems to exchange dependency and licensing data reliably.

3.5.2 Interoperability trade-offs

Standards such as SARIF and SPDX address specific domains but do not cover multiple evidence types in a unified manner [34]. Custom canonical schemas offer the flexibility to aggregate heterogeneous evidence and apply deterministic hashing but may sacrifice compatibility with existing tools. These concepts appear in prototype tools and research discussions but have not yet been formalized as standards.

3.6 Synthesis

Prior works highlight several gaps:

Evidence collection often occurs post facto and does not enforce gating within the pipeline.
Attestation frameworks typically focus on build provenance, with less emphasis on unifying multiple verification domains.

Policy-as-code systems largely operate on static configuration rather than dynamic verification evidence.

Existing standards address specific artifact types but do not support heterogeneous, hash-verifiable reports spanning multiple domains.

From a technical standpoint, existing frameworks provide isolated advances—canonical serialization for cryptographic assurance, provenance schemas for artifact traceability, and policy engines for enforcement. However, no unified architecture yet integrates these dimensions into a single verifiable workflow. This thesis addresses that gap by proposing a canonical evidence model and enforcement mechanism that operationalize these principles within continuous integration environments.

4 Methods and Implementation

This chapter describes the realization of the conceptual design as a functional prototype. It outlines the methodological approach and then details the concrete software artifacts that implement the automated verification system.

4.1 Method

The verification of a software projects is formalized as a repeatable, policy-enforceable process.

Objective evidence: Derive objective evidence about a repository's build hygiene and supply-chain posture.
Customized Validation: The governer can define optional and required checks.

The process combines two phases:

Measurement: Multi-aspect analysis of a repository.
Attestation: Policy based evaluation and decision.

To operationalize this method, a set of goals defines its expected behavior and output characteristics. These goals provide the criteria by which the method's correctness and reproducibility can be assessed.

4.1.1 Goals

It process aims to achieve an end to end verification and validation of a software project, with a focus on the following two aspects

Repeatable verification and validation. The verification process must be deterministic. Given a Go repository at a specific revision, it should always reproduce the same assessment under comparable conditions. This requires controlled inputs (such as a fixed revision and containerized runtime), a deterministic check order, and explicit handling of non-determinism (for example, by excluding timestamps from hashed material).

Standardized, policy-consumable JSON report. Results are collected in a stable JSON schema. The document must be expressive enough for a policy system to reason about statuses, counts, severities, and provenance signals without re-executing checks.

The schema separates metadata (context) from measured facts, so integrity protection is scoped to the latter.

Cryptographic integrity. Cryptographic substantiation ensures that the measured content can be checked for tampering at a later stage. A canonicalized-digest pipeline is applied: the report section is rendered into a canonical string (with fixed key ordering, explicit whitespace rules, and ordered lists where required) and then hashed using a collision-resistant function.

While these goals define the desired characteristics of the system, their realization requires addressing several challenges, discussed in the following subsection.

4.1.2 Problem Analysis

To paint a clearer picture of the problems that need to be solved, the following considerations were taken into account in the design of the verification service:

Trust boundary: Measurement engine and the policy engine are separated, the policy engine must not implicitly trust the measurement engine. Instead, it validates integrity before evaluating rules on the attested content.

Determinism vs. liveness: Some checks depend on mutable external knowledge (e.g., vulnerability advisories), To maintain repeatability, the measurement captures observed states as data (identifiers, affected components, fixed versions) rather than re-querying external sources at policy evaluation time.

Granularity of evidence: Each check produces both a coarse status and detailed structured results. This allows policies to express constraints at different levels (e.g., "no failures" or "no findings above severity threshold X") and supports future re-scoring without schema changes.

Attack surface: Risks include report rewriting, omission of results, and reordering during canonicalization.

A precisely defined canonical form mitigates these by fixing serialization semantics and hashing only the semantically relevant region.

Based on the identified challenges, a solution was devised that balances determinism, trust separation, and integrity assurance.

4.1.3 Search for the right solution

A solution was sought keeping in mind the following considerations:

Representation. For our purposes, JSON fits in the requirements and is a common format for the ecosystem.

Integrity mechanism. Hashing is applied to the canonicalized report region rather than the entire document. This allows metadata fields (e.g., timestamps, run identifiers) to change without breaking integrity of measured facts.

Policy enforcement. A declarative policy model evaluates the JSON artifact. A helper function supplies canonical rendering inside the policy context, ensuring that recomputed digests match the semantics of the report generator.

After setting up the methodological bases and operational sequencing, the chapters of this part explain the internal design and implementation of the verification service The implementation follows a layered and deterministic approach designed to create

reproducible verification outcomes. Each component was built to contribute to a structured pipeline that begins with evidence collection and ends with policy-based validation.

At the core of this approach is the **verification service**, which transforms a source repository into a consistent compliance artifact. It operates through a command-line interface that interprets user input, resolves configuration, and executes a predefined sequence of verification modules. Every run is isolated within a controlled workspace to remove environmental variation and guarantee that identical inputs always produce identical yet uniquely identifiable results.

Verification service is followed by the **policy evaluation layer**, implemented using Open Policy Agent (OPA) and a custom plugin. This custom plugin extends OPA to support hash verification and canonical report serialization, thereby introducing cryptographic integrity into the policy decision workflow. Policies are defined declaratively to ensure that identical verification evidence consistently results in the same compliance outcome.

The overall system is integrated into a **continuous integration (CI) pipeline** that coordinates the complete compliance validation process. This pipeline consists of two structured phases that operate together to achieve end-to-end artifact verification:

Phase 1: Build and Verification This phase generates release artifacts along with detailed supply chain metadata. It records the full build environment, dependencies, and transformation history required for subsequent verification. These produced artifacts are then subjected to a defined set of checks, and a structured report is generated.

Phase 2: Policy Evaluation This phase processes the report from Phase 1 and enforces compliance using the custom OPA plugin and Rego-based policies. The CI gate reflects the final verification result and determines whether the build is marked as failed or permitted to proceed to deployment.

A layered design of verification, policy evaluation, and automation ensures determinism, transparency, and reproducibility throughout the entire compliance validation lifecycle.

4.2 Verification service: Design and Implementation

In this section we describe in detail the reasoning behind the design decisions and structure of the verification service.

This system is designed around a structured verification pipeline that transforms a source repository into a reproducible compliance artifact. At the highest level, the control surface is a command-line interface (Command Line Interface (CLI)). Its role is to interpret user intent (governor in this case) by parsing flags and the target repository reference.

From this input, the CLI produces a run configuration, which specifies three essential elements, a more visual representation is shown in Figure 4.1:

The scope of analysis: which repository and which subsets of checks to execute
Execution parameters: flags, modes, or exclusions
output preferences: persistence, streaming, formatting

The design goal at this stage is determinism. Each configuration produces an unambiguous plan that can be executed again under identical conditions, leading to reproducible outcomes. Once the input and configuration are validated, the target repository is cloned into a controlled workspace.

This workspace is isolated and created fresh for each run so that no residual state or cached files affect the analysis. This isolation ensures that each run operates on a snapshot identical to the repository's reference and simplifies reasoning about determinism because external noise is excluded.

This system under examination is composed of modular components, each of which fulfills a distinct verification or quality-assurance function. At a conceptual level, the modules can be categorized into

Coordination and orchestration: 4.2.1
Static code analysis: 4.2.3
Formatting enforcement: 4.2.4
Vulnerability detection: 4.2.5
Commit authorship verification: 4.2.6
Manifest scanning: 4.2.7
Provenance verification: 4.2.8
Aggregation of results: 4.2.10

The aggregation of results across these modules leads to a unified report whose status can be interpreted in binary form (PASSED or FAILED), with additional intermediate states (e.g., WARNING, SKIPPED) serving as qualifiers.

A core principle to follow is to keep the report as strict as possible, so that the policy engine can be configured with the most possible degree of flexibility. For example, if a check is skipped, it should be marked as SKIPPED in the report, and if a check fails, it should be marked as FAILED. This way, the policy engine can be configured to allow or deny certain checks based on the report.

Another core principle is to keep the report as comprehensive as possible, so that the later stages have as much information as possible, but the report should still remain human readable.

4.2.1 Coordination Layer

Coordination layer of the verification system is defined in the program's entry point, `main.go`. The `main` function serves as the central orchestrator, interpreting user intent, initializing the verification workflow, and managing the lifecycle of each verification run.

Execution begins with the parsing of command-line arguments and configuration flags provided by the user. These inputs specify the analysis parameters, including the target repository, enabled or disabled checks, execution modes, and output preferences. The parsing stage validates and transforms the input into a structured configuration object that directs all subsequent operations. After configuration, the function clones the target repository into an isolated workspace, ensuring reproducibility and preventing interference from external states or prior runs.

Once the repository is prepared, the `main` function triggers the core report generation routine. This routine collects and aggregates the outputs from individual verification modules into a single, unified report structure. Upon completion, the coordination layer ensures the report is stored persistently, either on disk or another specified output location. Optional routines may also be executed to render results in a human-readable form, such as console output or summarized statistics.

A high-level abstraction of the `main` function's logic is as follows, detailed in Section 4.2.10:

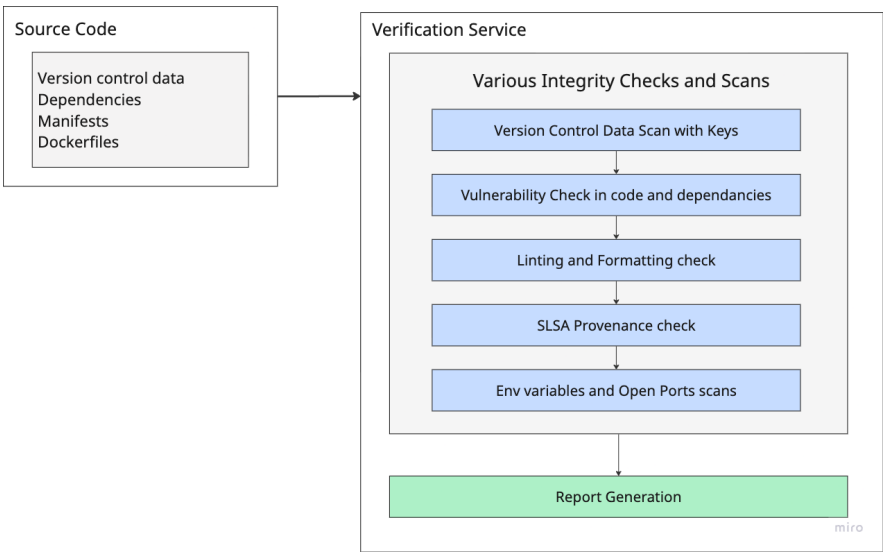

Figure 4.1: High level design of verification service

```
function main(flags):
    input ← parseFlags(flags)
    repo ← cloneRepository(input.target)
    report ← GenerateReport(repo, input)
    saveReport(report)
    prettyPrint(report)
```

Here, the flags determine which checks to run and which ones to skip, and also control if the output should be persisted to disk or printed to the console.

Crucially, the coordination layer is also responsible for ensuring the determinism of the verification process. In summary, the `main` function provides a disciplined entry point that structures the verification workflow, delegates specialized tasks, and upholds the system's guarantees of reproducibility and helps in maintaining integrity.

4.2.2 Report Construction

The report package defines the central data structures (ReportData, Report). Conceptual considerations enforce that each check must produce a status and optional metadata and the aggregation function must reduce multiple check outcomes into a single overall result.

This can be modeled as follows: Let $R = \{c_1, c_2, \ldots, c_n\}$ be the set of all check results, where $status(c_i) \in \{\text{PASSED}, \text{WARNING}, \text{SKIPPED}, \text{FAILED}\}$.

$$overallStatus(R) = \begin{cases} \text{FAILED}, & \text{if } \exists c_i \in R : status(c_i) = \text{FAILED} \\ \text{FAILED}, & \text{if } \exists c_i \in R : status(c_i) = \text{SKIPPED} \\ \text{PASSED}, & \text{otherwise.} \end{cases}$$

Stability of the report output is ensured through deterministic hashing that removes irrelevant variations such as whitespace or key ordering.

4.2.3 Static Analysis and Linting

The linting module represents a category of checks aimed at code correctness and maintainability. It models two levels of severity:

Errors detected by semantic analysis tools (e.g., type checks, vetting).
Warnings flagged by stylistic or heuristic linting tools.

Abstract evaluation:

> The linting process first runs semantic checks on the source code to find errors. Then it runs stylistic linting to detect warnings. If any errors are found, the process outcome is `FAILED`. If no errors but some warnings are found, the outcome is `WARNING`. If neither errors nor warnings are found, the outcome is `PASSED`.

The rationale to lint the code is to identify issues without modifying source files, thus preserving the integrity of the analyzed repository.

4.2.4 Formatting Verification

The formatting module enforces conformity with canonical language formatting standards. Its abstraction is straightforward:

> Formatting module checks the source code for files that do not follow the required formatting standard. If any unformatted files are found, the process outcome is `FAILED`. If all files conform to the standard, the outcome is `PASSED`.

The rationale to format the code is to ensure that if the code follows the formatting standards, if changed in a way that is not intended by the developer, the module outcome is `FAILED`.

4.2.5 Vulnerability Scanning

The vulnerability scanning module introduces external knowledge into the verification process by comparing the code and its dependencies against a vulnerability database to identify known vulnerabilities.
Conceptually:

> Vulnerability scanning process queries the Open Source Vulnerability (Open Source Vulnerabilities (OSV)) database using the project's dependencies. If any vulnerabilities are found, the module outcome is `FAILED`. The found vulnerabilities are added to the report. If no vulnerabilities are detected, the outcome is `PASSED`.

Findings are enriched with severity metrics (e.g., Common Vulnerability Scoring System (CVSS))[6] to support prioritization.

4.2.6 Commit Authorship Verification

Commit authorship verification module enforces trust in code history by checking cryptographic signatures.
Conceptually:

The module is supplied with a set of allowed GNU Privacy Guard (GPG) keys, it first loads all the keys. It then retrieves the commit log from the repository, including the signature status and key ID of each commit. For every commit, the module checks whether the signing key matches one of the allowed or imported keys. Commits signed with valid keys are counted as verified, while unsigned or improperly signed commits are marked as unverified. The final status is `PASSED` only if all commits are verified; otherwise, it is `FAILED`. If the repository contains no commits, the result is also `FAILED`.

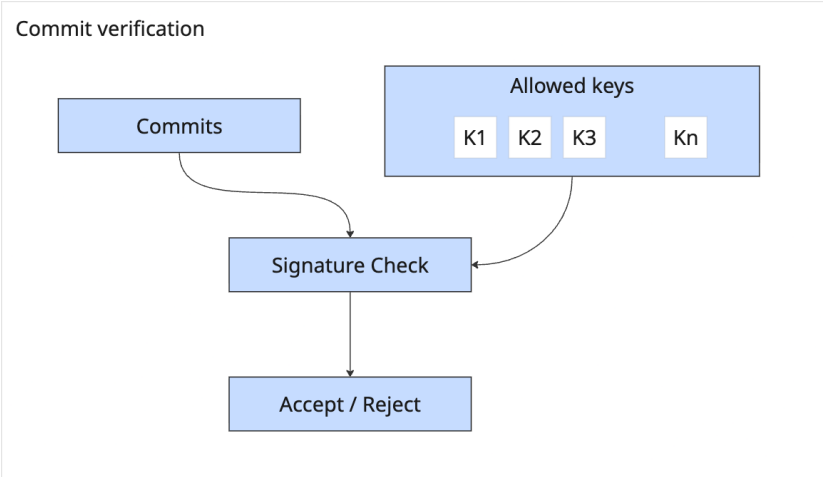

Figure 4.2: Commit signature verification against allowed keys

This process ensures that every commit originates from an authorized key and that any unsigned or invalid signatures are clearly reported.

4.2.7 Manifest File Checks: Open Ports and Environment Variables

The manifest module identifies and gathers information about open ports and environment variables declared in Kubernetes manifests and Dockerfiles found in the source code.

Conceptually:

> The module scans all files in the repository to detect declared environment variables and open ports, and collected data is stored for later analysis. At this stage, only data collection occurs; validation against allowed ports and environment variables is performed later by the policy engine.

4.2.8 Provenance Verification

The slsa module validates artifacts against the SLSA (Supply-chain Levels for Software Artifacts) standard. This check is based on provenance records and binary verification. This design ensures reproducible builds and resistance to tampering.

Conceptual Basis. The verification process consumes provenance statements expressed in the `in-toto` or `SLSA Provenance` format. Each statement contains metadata describing the builder identity, input sources, build parameters, and the resulting artifact digests. The verifier ensures that the provenance document is syntactically valid, and all declared inputs (e.g., commit hash, dependencies) match the actual repository state, and the digest of the built artifact corresponds to the value declared in the provenance record.

4.2.9 Manifest Scanning

Manifest scanning module analyzes the repository to extract configuration and deployment information from Dockerfiles and Kubernetes manifests.

Conceptually:

> Scanning process searches the repository for Dockerfiles and YAML Ain't Markup Language (YAML) files that may represent Kubernetes manifests. Each Dockerfile is examined for exposed ports and environment variables using pattern matching on `EXPOSE` and `ENV` directives. Kubernetes manifests are parsed to verify structure and extract metadata such as resource kind, name, ports, and environment variables from container specifications. Files that cannot be parsed or contain invalid data are flagged as having issues. If any file has issues, the module reports a `WARNING` status; otherwise, the result is `PASSED`.

This module provides structured data about deployment configurations to support later policy validation.

4.2.10 Aggregation Logic and Report generation

The overall aggregation follows strict precedence rule: "if any check fails or is skipped, the overall outcome is marked as FAILED; only when all checks pass is the overall status recorded as PASSED."

Since the status fields are more geared towards humans to consume, the aggregation logic is kept simple. A reporting component collects the outputs of all checks and assembles them into a structured JSON report. This generated report is then used to evaluate the compliance of the repository against the defined policies by the policy engine.

4.2.11 Continuous integration context

When integrated into a continuous integration (CI) environment, the report becomes the input to a policy enforcement stage. A policy engine re-computes the canonical form, verifies the digest, and evaluates compliance rules defined by the organization. The final CI decision depends on two dimensions:

Integrity whether the report content matches its digest.

Compliance whether the findings satisfy the declared policy (for example, which warnings are tolerated, which failures are blocking).

This split enforces the logic: the verification pipeline produces evidence, the digest ensures the evidence is intact, and the OPA system applies organizational governance rules over that evidence.

The pseudocode outlines the workflow of the verification process. It begins by cloning the target repository and then executes a series of selected checks, depending on the flags. Results are then collected, and skipped checks are noted. An overall verification status is then derived and a structured report is generated.

Pseudocode .

```
INPUT: repo_url, flags
```

```
clone(repo_url) -> workdir
results := {}

for check in [lint, format, vuln, commits, env, ports, slsa]:
    if enabled(check, flags):
        results[check] := run_check(workdir)
    else:
        results[check] := {status:"SKIPPED"}

overall := reduce_status(results)
report_hash := sha256(canonical_string(report))

output := {
  metadata: {
    project_name, repo_url, commit_hash, commit_message,
    checked_at, verifier_version, run_id,
    verification_status: overall,
    report_sha256: report_hash
  },
  report: { ...results... }
}
```

The implementation code for the verification service is available on GitHub[79].

Once the verification report is generated, the next step is to evaluate it against defined compliance policies. This is achieved through the integration of the OPA framework.

4.3 Open Policy Agent Integration

Having established the methodological foundation and operational flow, the following section details the internal design and implementation of the verification service, the core component that realizes these concepts in practice.

4.3.1 Process Flow

To address the requirements, the compliance policy is divided into two parts:

Config is where the user defines which parts of the verification should be strictly considered for validation. Specific allowed ports, and allowed environment variables can be configured in this section.

Policy decisions section defines how to handle the report generated by the verification service based on the config defined prior.

The separation of roles between these two layers was intentional. This division enforces modularity, simplifies extensibility, and ensures that config and policy remain declarative and adaptable to future changes.

4.4 Custom OPA Plugin

While OPA provides general-purpose policy evaluation, its built-in functions were insufficient for this work's requirements. Specifically, the SHA-256 hash calculation algorithm

did not work similarly to other implementations (golang and bash). A custom plugin was therefore implemented to extend OPA's capabilities.

The purpose of said plugin also serves as a proof of concept that the Rego policy can be extended beyond Rego in another language like Golang.

Conceptually, the plugin performs four tasks:

Canonical Serialization : Produces a stable string representation of the report by enforcing sorted keys and compact JSON formatting.

Hash Verification : Calculates the Secure Hash Algorithm (SHA)-256 value of the canonicalized report and compares it against the stored hash.

Custom Builtin Functions : Provides a callable interface within Rego policies for report inspection.

Error Handling : Returns structured error objects for missing or inconsistent data.

A simplified pseudocode description of the integrity verification step is as follows:

```
flat_report = canonicalize(input.report)
calculated = sha256(flat_report)

if stored_hash is missing:
    deny("metadata.report_sha256 is missing")
else if calculated != stored_hash:
    deny("hash mismatch")
else:
    allow()
```

The full source code for the plugin is available on GitHub[73].

This mechanism introduces cryptographic guarantees into the policy layer.

The plugin therefore transforms OPA from a pure policy engine into a tamper-aware verification framework.

To enable the policy engine to interpret verification outputs consistently, a structured and deterministic JSON report format was defined.

4.4.1 JSON Report Structure

At the core of the system lies the JSON schema used for report generation and interpretation. The design of the schema followed two main principles: deterministic integrity and uniform policy evaluation.

The report structure can be summarized as follows:

Metadata Block : Contains the stored SHA-256 hash and other identifying details.

Domain Sections : Each verification domain is represented with a standardized result format.

Status Encoding : Each section reports status values such as "PASSED", "FAILED", or "WARNING", thereby enabling consistent parsing.

Failure Aggregation : All failures are recorded in a dedicated object for uniform policy-based processing.

Summary Block : Includes calculated hash values, hash match status, required check configuration, and the final "PASSED" or "FAILED" outcome.

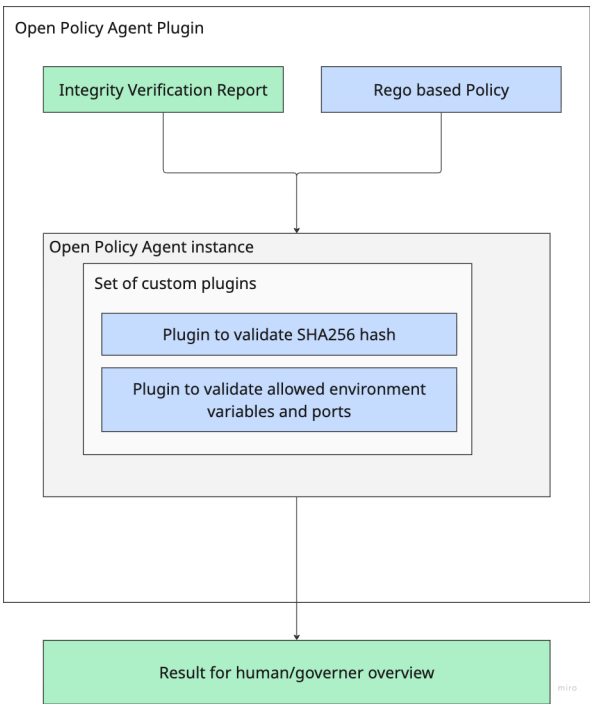

Figure 4.3: Policy Evaluation based on custom rego policy and OPA plugins

This structure ensured that the report could be both machine-readable and easier to understand by humans. Deterministic serialization, achieved through sorted keys and whitespace removal, guaranteed that the same logical report would always yield the same hash value.

4.4.2 Policy Evaluation Layer

The Rego-based policy layer defines the actual decision rules. Its conceptual design rests on three ideas:

Declarative constraints : Policies define the desired state (e.g., "all required checks must succeed") rather than procedural logic.

Unified enforcement : Failures across multiple domains are aggregated into a single outcome, thereby preventing fragmented interpretations.

Configurable strictness : Warning results may be escalated to failures depending on policy configuration, supporting different operational needs.

The policies include rules for hash verification, requirement enforcement, and overall compliance evaluation, with required checks such as linting, formatting, environment variable scanning, and SLSA verification always enforced, while optional checks like vulnerability scanning or Dockerfile scanning may be activated or deactivated according to configuration.

The integration of verification and policy evaluation components reaches its full potential within an automated continuous CI pipeline, where these mechanisms operate as sequential, verifiable stages.

4.5 Continuous Integration Automation

CI orchestration was implemented as a structured pipeline enforcing a strict separation between artifact construction, provenance generation, and policy-based adjudication. Each

stage executes within an isolated environment to avoid shared state and to guarantee that all verification outcomes are grounded on reproducible inputs rather than mutable host conditions.

4.5.1 Method Outline

The pipeline was modeled as a directed sequence of two major phases. Each phase consumes the verified outputs of the previous one and produces a well-defined artifact for the next. This conceptual separation aligns with the principles of determinism, verifiable provenance, and minimal trust.

The process is divided into:

Build and Provenance Generation : focuses on compiling the source and attaching verifiable provenance metadata.

Verification and Policy Evaluation : validates the generated evidence against declarative policies to determine final acceptance.

Figures 4.4 and 4.5 illustrate these two phases as separate Unified Modeling Language (UML) diagrams for clarity and readability. This division replaces the earlier single large diagram to provide a clearer vertical representation suitable for the thesis layout.

4.5.2 Phase 1: Build and Provenance Generation

Source code is retrieved and metadata such as commit identifiers and version information are computed.

The project is compiled in a controlled build environment.

A provenance record describing the build context, inputs, and outputs is generated and stored with the produced binary.

This phase concludes with immutable artifacts: the compiled binary and its provenance metadata.

4.5.3 Phase 2: Verification and Policy Evaluation

The verification service retrieves the artifacts and reconstructs the evaluation environment.

A verifier process replays integrity checks and produces a structured verification report.

A policy engine evaluates this report against declarative rules (e.g., dependency hygiene, signature presence, or configuration compliance).

The resulting decision specifies both the pass/fail outcome and explicit reasons for rejection when applicable.

Final outputs include the verified artifacts, provenance records, and policy evaluation results, which together define the reproducible CI evidence chain.

4.5.4 Gate

The overall CI status is derived from the policy evaluation results.

Result of the policy evaluation is used as a signal by the Governever to decide if the build is allowed to proceed to the next stage.

4.6 Summary

In summary, this section presented the design and implementation of the verification system, detailing its architecture, operational flow, and integration within a continuous integration pipeline. Through layered construction-comprising evidence generation, canonicalized

reporting, cryptographic attestation, and policy-based validation-the prototype demonstrates how compliance verification can be made deterministic, reproducible, and tamper-evident. The modular design enables extensibility while preserving traceability and consistency across runs.

Our next chapter evaluates this implementation by examining its performance, scalability, and robustness under varying repository sizes and configurations, thereby validating the practical soundness of the proposed approach.

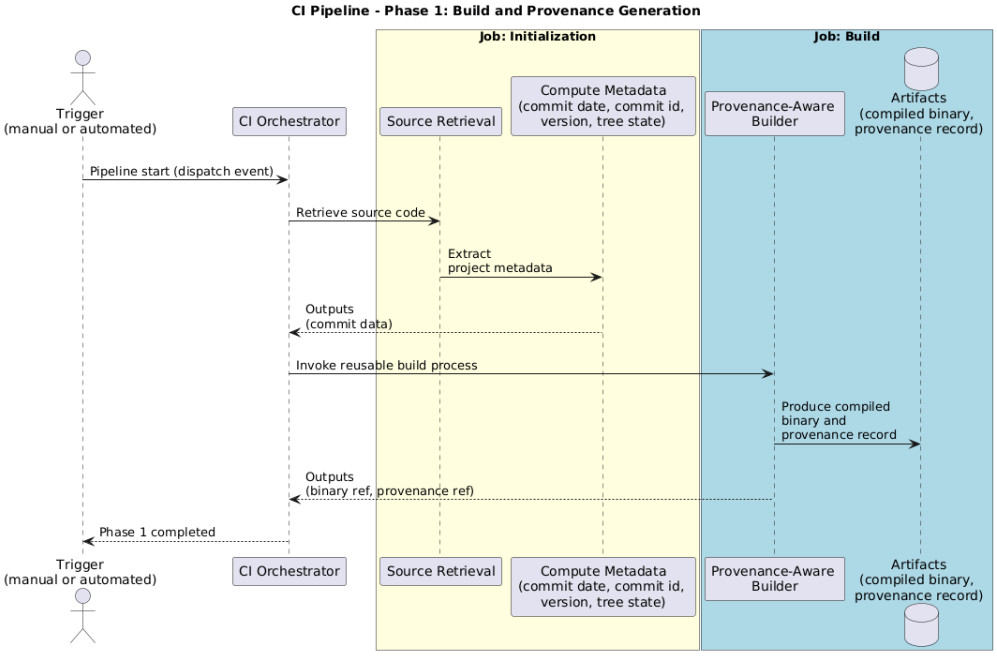

Figure 4.4: CI Pipeline – Phase 1: Build and Provenance Generation

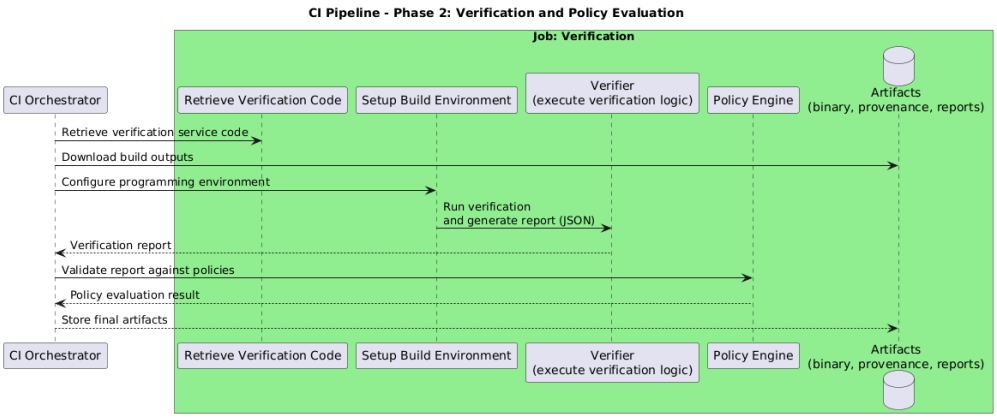

Figure 4.5: CI Pipeline – Phase 2: Verification and Policy Evaluation

5 Results

5.1 Evaluation and Results

This section presents the results of the evaluation. The goal of the evaluation is to assess the correctness, scalability, and practical usability of the verification system. Results are structured along quantitative measures (execution time, resource usage, consistency of outputs) and qualitative aspects (integration effort, clarity of artifacts, observed peculiarities). Figures and tables are used where appropriate to support clarity of presentation. Interpretation of these results is left to the following discussion section.

5.1.1 Evaluation Setup

Three Go-based repositories of different scales were selected in order to observe the behavior of the tool under increasing size and complexity: To keep the results reproducible, they were forked from the original repositories and the code was cloned locally.

go_proj : a basic Go project with 68 lines of code and 39 commits[69].
caddy : a mid-sized web server project with ∼84,229 lines of code and 2,359 commits[67].
kubernetes : a very large infrastructure repository with ∼4.8 million lines of code and 132,000 commits[68].

All experiments were executed on a machine with an Intel Core i7–10710U (12 logical CPUs) and 62 GiB Random Access Memory (RAM), unless otherwise noted. Each repository was evaluated five times to account for variability in runtime and to obtain stable average measurements. In addition to local evaluation, experiments were repeated on GitHub Actions using the hosted CI runners (4 vCPU, 16 GiB RAM) to validate real-world usability in continuous integration workflows.

Having established the experimental conditions and chosen repositories, the next section presents the quantitative performance results, focusing on runtime and scalability across projects of increasing size.

5.1.2 Quantitative Results: Execution Time on Local Hardware

The verification pipeline was executed five times per repository. The resulting execution times are shown in Table 5.1.

Table 5.1: Execution time across repositories (local machine, 5 runs each)

Repository	Avg runtime	Min runtime	Max runtime
go_proj	∼6.7 s	5.2 s	12.0 s
caddy	∼16.2 s	15.4 s	17.8 s
kubernetes	∼1769 s (≈29 min)	1748 s	1784 s

Description: This chart compares three repositories: `go_proj`, `caddy`, and `kubernetes`, using three metrics:

Lines of code (Lines of Code (LOC)): Extracted directly from each repository using standard code counting tools.

Commits: Counted by analyzing the Git version control history of each repository.

Runtime: Each repository was processed five times using the same verification tool under identical conditions. The reported runtime in both charts corresponds to the arithmetic mean of these five runs. For reference: `go_proj` ≈ 6.7 s, `caddy` ≈ 16.2 s, `kubernetes` ≈ 1769 s.

The y-axis is plotted on a logarithmic scale. This scaling was chosen because the raw values differ by several orders of magnitude. Kubernetes, with over 4.8 million LOC and 132,000 commits, would otherwise overshadow the much smaller repositories. The log scale compresses the values in a mathematically consistent way, preserving proportional differences while making all repositories visible in the same chart.

As a note, the feature to determine the code coverage of the repository was not included in the evaluation, as it was added later to make the tool feature complete after the evaluation was conducted.

Data preparation and calculation

The data was prepared and calculated as follows:

The results of the runtime measurements can be seen in Figures 5.1 and 5.2. A logarithmic scale was chosen because both *Lines of Code* and *Number of Commits* span several orders of magnitude, and using a linear scale would compress small repositories such as `go_proj` (68 LOC, 39 commits) into an almost invisible range compared to large ones like `kubernetes` (over 4.8 million LOC, 132k commits).

In the diagrams, we can observe that runtime increases proportionally with repository complexity, whether measured by code size or commit count. The verifier completes within seconds for minimal repositories, requires tens of seconds for medium-sized projects such as `caddy`, and approaches half an hour for extensive systems like `kubernetes`. The roughly linear alignment in log-log space indicates near-polynomial scaling behavior between verification time and repository size.

While the overall scaling behavior is a global view of performance, a finer touch-up of internal timing per verification step indicates the point where computational effort builds up. The remainder of this analysis looks at these internal dynamics using the medium sized caddy repository as a representative case.

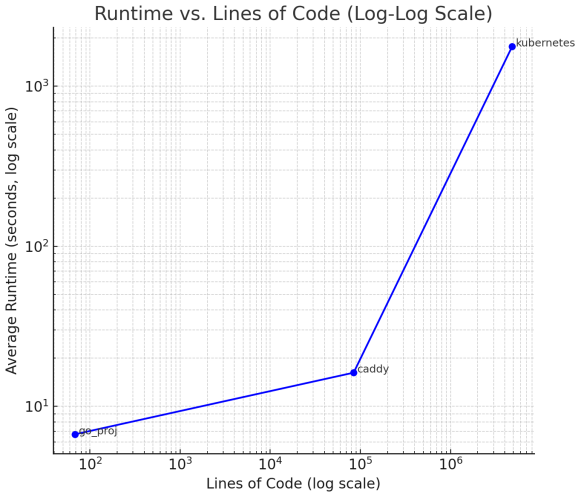

Figure 5.1: Runtime vs. Lines of Code (Log-Log Scale)

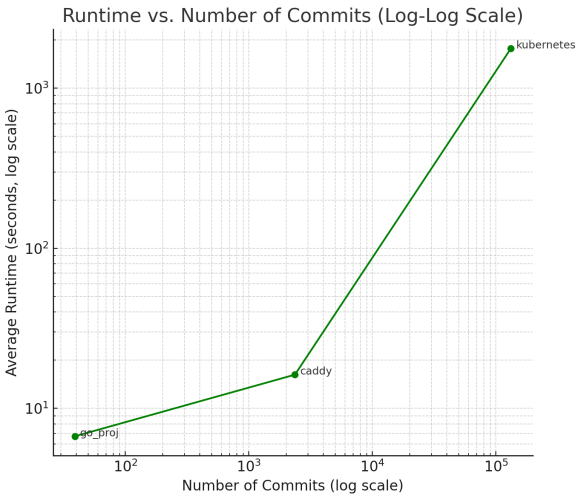

Figure 5.2: Runtime vs. Number of Commits (Log-Log Scale)

Detailed Timing Analysis for the Caddy Repository

To better understand internal performance, an additional timing analysis was performed on the `caddy` repository. The verification service was executed ten times, and detailed component-level timings were collected for each run.

Each verification step in the service is wrapped by a lightweight timing function that records the start and end of execution. The duration is computed as the difference between the two timestamps and written into the structured report alongside functional results. This mechanism is implemented uniformly across all verification components, so that every run produces a detailed timing profile without modifying the core logic. All timing data are serialized into a new section in the JSON report which can conveniently be ignored if not needed.

Figure 5.3 shows the average duration per verification step with corresponding standard deviations, while Figure 5.4 displays the total runtime variation across the ten runs.

A separate analysis script can then parse these JSON logs to compute statistical summaries such as mean duration, variance, and per-step contribution to total runtime.

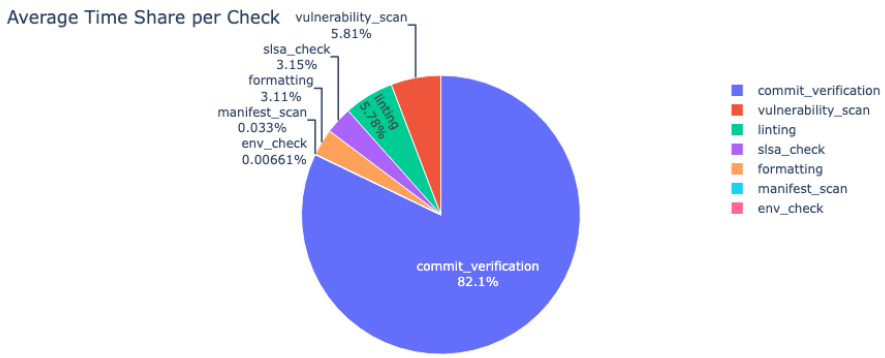

Figure 5.3: Average duration per verification step (Caddy repository, 10 runs)

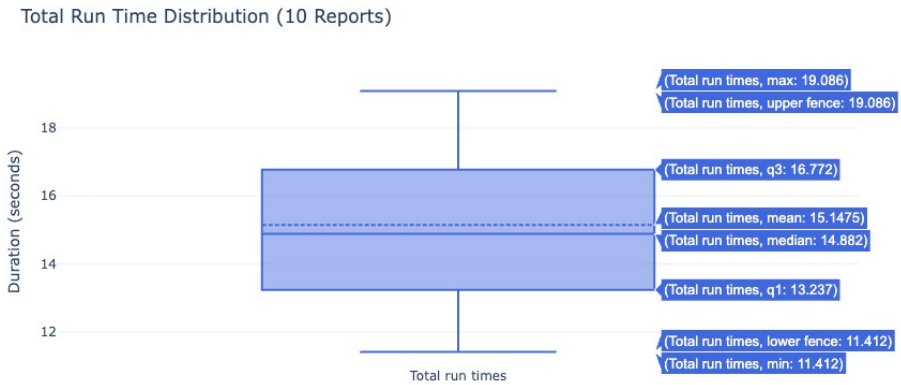

Figure 5.4: Total runtime per report (Caddy repository, 10 runs)

Results: The total runtime across all runs averaged 15.15 s (±2.53 s). The `commit_verification` step dominated execution time, taking 12.43 s on average (82% of total). All other checks each contributed less than 6%. Among these, `vulnerability_scan` and `linting` were the next most time-consuming at roughly 0.88 s each. The slowest individual run took 19.09 s, while the fastest completed in 11.41 s, indicating moderate variability across executions.

Interpretation: The results confirm that verification performance for a medium-scale project such as `caddy` remains well within interactive limits. The high share of time in `commit_verification` suggests that the number of commits in the repository affects the execution time severely. Another possible explanation is that since Git uses GnuPG to check that the commit's signature matches one of the provided public keys, and the cryptography used in GPG is asymmetric encryption with digital signatures based on OpenPGP (RSA or Ed25519 + SHA-256), the verification of the commit signature is a computationally expensive operation.

Other phases (`slsa_check`, `formatting`, `linting`) show low runtime and high consistency, reflecting predictable overhead. Variability across runs is limited and mainly linked to network-bound operations during vulnerability scanning. Overall, these measurements indicate that most of the total runtime stems from a single component and that further optimization efforts should focus on reducing its cost.

Conclusion: Since, the choice of caddy, which is a repository with limited diversity of code, most of the performance is hurt by the large number of commits. An analysis with a more diverse codebase would be needed to test the performance of each section of the verification pipeline.

Beyond runtime behavior, it is also important to understand the structure of the produced verification outputs, as these artifacts form the basis for reproducibility and downstream policy evaluation.

Validation Output Structure

The custom plugin produces a standardized JSON output for every run. The structure of the JSON validation output is illustrated in Section 5.1.2. For a complete example and additional details, refer to Appendix A.

Listing 5.1: Structure of JSON validation report, for full example see Appendix A

```
{
  "metadata": {
    ...
  },
  "report": {
    "formatting": {
      "status": "PASSED",
      "details": {...}
    },
    "linting": {
      "status": "PASSED",
      "details": {...}
    },
    "vulnerability_scan": {
      "status": "PASSED",
      "findings": [...],
    },
    "commit_verification": {
      "status": "PASSED"  ,
      "findings": [...],
    },
    "env_manifest": {
      "status": "PASSED",
      "open_ports": [],
      "env_vars": [...],
    },
    ...
  }
}
```

The output shows the verified digest, integrity match, and Boolean outcomes for each required policy dimension. The deterministic nature of the digest computation ensures that repeated runs on identical input yield byte-identical results.

After validation on the deterministic character of local outputs the next step is testing how the verifier functions with real continuous integration situations to catch how the verifier operate in hosted environments.

CI/CD Execution Times

To evaluate practical feasibility, the full verification pipeline (verifier + OPA validation + provenance generation) was executed in GitHub Actions. Due to runner constraints, only the smaller and mid-sized repositories were tested. Results are summarized in the Table 5.2.

Table 5.2: Execution time on GitHub Actions (5 runs each)

Repository	Min runtime	Max runtime
go_proj	2m 48s	3m 20s
caddy	4m 32s	4m 41s

The additional overhead compared to local execution results primarily from container startup, provenance generation, and CI job orchestration.

5.1.3 Qualitative Results

To complement the quantitative evaluation, qualitative aspects were assessed to understand how the system performs in real development environments and how easily it can be integrated into existing workflows.

Integration Effort

Integration was evaluated by setting up the verification system in existing CI pipelines. Configuration consisted of two steps: (i) defining required policy in OPA, and (ii) adding a single job to the repository's GitHub Actions workflow. The setup required minimal setup time and no invasive changes to existing build logic, making adoption straightforward for new projects.

Observed Behavior

The following observations summarize how the system performed across repositories of varying complexity and size, highlighting both consistency and scalability characteristics.

Small repositories : Reports were generated quickly and consistently. Digest values matched across runs, confirming determinism of canonicalization.

Medium repositories : Execution times remained acceptable ($<$20 s locally, $<$5 min in CI). The JSON output was stable and integrated cleanly with OPA policy evaluation.

Extremely Large repositories : For `kubernetes`, execution exceeded 29 min. Hence, it revealed practical limits of the current sequential execution model. Although the outputs were correct, the latency made the system less suitable for extremely large repositories in its current form.

The results of this combination of quantitative and qualitative results describe the verifier's technical performance and integration properties. To test how to make these verification artifacts and policies behave in a real-life workflow, in a real-life GitHub Actions environment, the next section shows how to enforce such policies.

5.2 Policy-as-Code Evaluation Demonstration on GitHub

After validating the system's behavior across different project scales, the next step demonstrates how these verification and policy mechanisms operate in a real CI/CD environment.

5.2.1 Overview

The section presents the experimental setup and demonstration of automated policy enforcement through the integration of GitHub Actions and the Open Policy Agent (OPA). The objective was to verify that software release compliance can be governed using policies defined and executed as code within the repository itself.

Two GitHub accounts were used to emulate realistic development governance roles:

Governor account: Has administrative privileges and can override merge protections when needed. Is able to set rulesets, branch protection rules, and other repository-level settings. Can modify the rego policy (where the config resides).

Developer account: Performs regular development activities such as creating pull requests, subject to automated checks and policy validation. Has very limited permissions, can not modify the rego policy.

As the caddy repository did not contain any kubernetes manifests and environment files, it was not ideal for this demonstration, rather a simple web server implementation in Go was created to use as the *target repository*. To trigger all the checks, files related toall the parts of the pipeline were added to the repository. It includes a `policy.rego` file defining OPA compliance rules and a GitHub Actions workflow file containing the complete CI/CD pipeline. The repository structure is shown in Figure 5.5.

5.2.2 GitHub Configuration

To enforce the verification results, GitHub's *branch protection rules* were configured on the main branch of the target repository, a small go web server project[69]. These ensure that certain branches require certain checks to run, and only pull requests passing all required checks can be merged. These rules are configured by the governor.

Branch protection: Only pull requests can update the main branch.
Required approvals: At least one approving review before merge.
Status checks: The check named `verify_and_validate` must pass before merging.

5.2.3 Workflow Structure

The GitHub Actions workflow was designed to run automatically on every pull request. It follows a three-stage process:

Build and Provenance: The Go project is compiled and provenance metadata is generated.
Verification: The verification service produces canonical JSON reports with cryptographic hashes.
Policy Evaluation: OPA evaluates the verification results against the defined texttpolicy.rego.

A simplified workflow example is shown below:

Listing 5.2: Simplified GitHub Actions workflow integrating build, verification, and policy evaluation.

```
jobs:
  verify_and_validate:
    runs-on: ubuntu-latest
    steps:
      - name: Generate provenance
```

```
1   .github
2       └── workflows
3           ├── main.yml
4           ├── show-build-path.yml
5           └── slsa-goreleaser.yml
6   policy
7       ├── ci.rego
8       ├── input_vul.json
9       └── input.json
10  .env
11  .slsa-goreleaser.yml
12  Dockerfile
13  go.mod
14  go.sum
15  index.html
16  main.go
17  osv-results.json
18  README.md
19  test_deployment.yml
```

Figure 5.5: Repository structure showing Go source files, `.github/workflows/`, and `policy.rego`.

```
      run: ./verifier --provenance
    - name: Run verification
      run: ./verifier --analyze --output verification.json
    - name: Evaluate policy
      run: opa eval --input verification.json --data policy.rego "data.verdict"
```

5.2.4 Policy Enforcement Scenarios

To illustrate how policy-as-code governs release verification outcomes, several representative enforcement scenarios were executed within the GitHub Actions pipeline.

Case 1: Multiple checks fail during verification

In the first scenario, the policy fails on multiple sections:

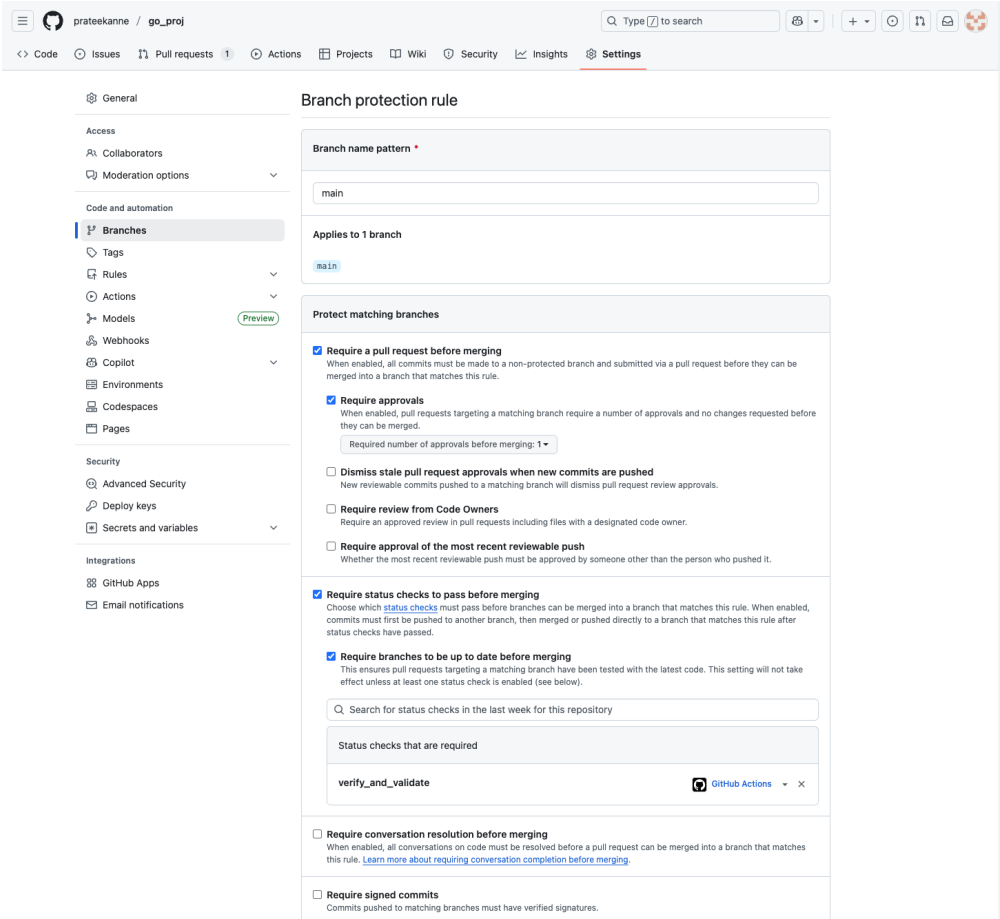

Figure 5.6: Branch protection rule configuration for the main branch with required approvals and verification checks.

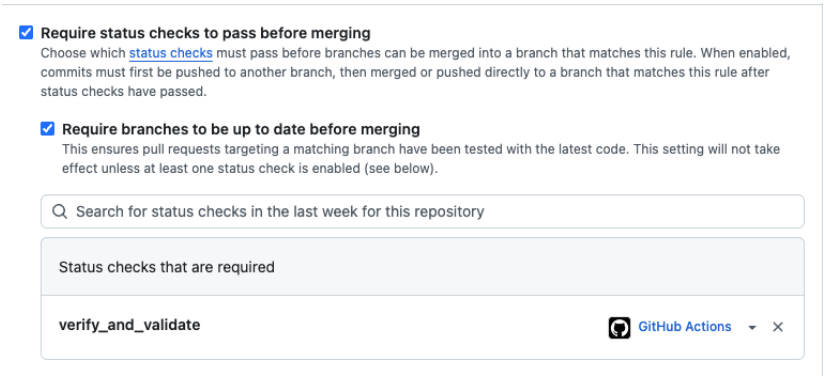

Figure 5.7: Status check configuration enforcing the `verify_and_validate` workflow as mandatory before merging.

Not all commits are verified and signed, but the commit verification check is set to required.
More environment variables are used than allowed, and the environment variable check is set to required.
A new port is used, which does not exist in the allowed list.

The developer commits changes without using a verified signature and adds code that does not comply with the policy. Which is then detected by the verification service and it flags the commit as unverified, and OPA discovers the violations including the extra port and environment variables, causing the OPA policy evaluation to fail. This is shown in Figure 5.8.

As a result, the GitHub Action fails at the `verify_and_validate` step, preventing the merge.

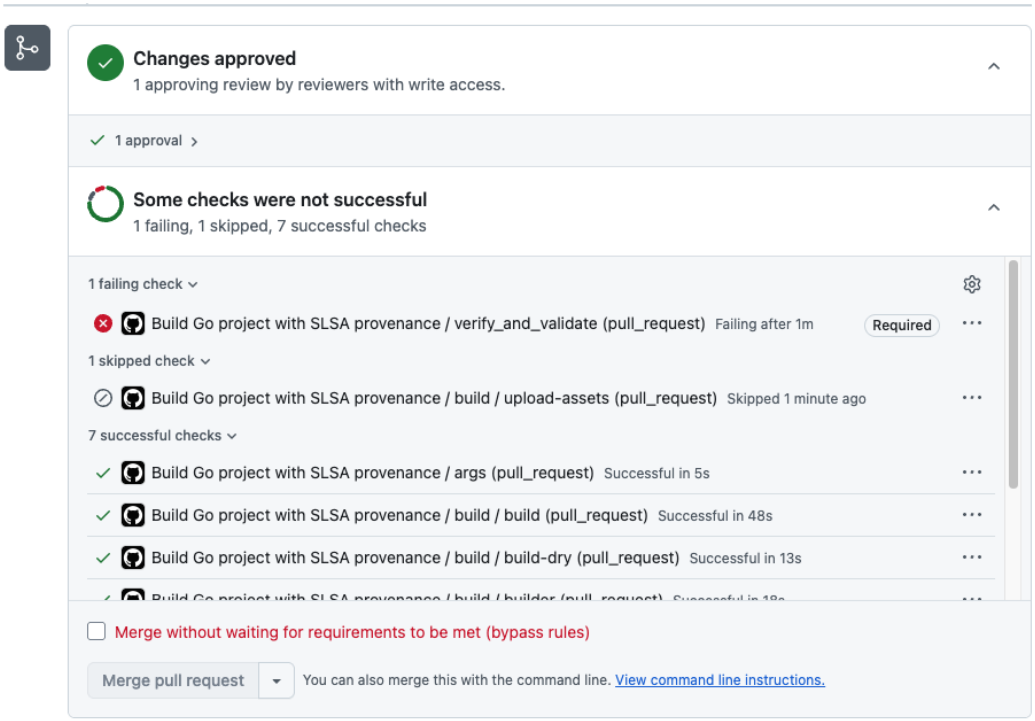

Figure 5.8: Pull request with failing `verify_and_validate` check due to unverified commit. The merge option is disabled for the developer.

The governor account, however, retains the ability to override this restriction by force pushing or using the *bypass rules* option, replicating the real-world governance where administrators can override automation when justified. This is shown in Figure 5.9.

Case 2: Policy Violation in Configuration (Port not allowed)

In the second case, the code introduces a service running on a port not defined in the existing policy configuration. The policy expects all ports to be explicitly approved in the Rego policy file. Since the new port is not mentioned in the policy, OPA evaluation complains about it as shown in line 5 of Listing A.4, the port is denied and the OPA evaluation fails, leading to a failed GitHub check and merge request is blocked.

To understand the failure, the governor can look at the logs of the failed pipeline run, where the OPA evaluation result is shown and the reason for the failure is analyzed. After analysis, the governor updates the policy to include the new port in the allowed list within `policy.rego`, and since the permission to modify the policy file is set in the CODEOWNERS file in the repo and permission is granted only to the governor, GitHub enforces this and only allows the governor to modify the policy. The next pipeline run passes all checks, and the pull request merges successfully. This shows the dynamic adaptability of policy governance.

Case 3: Successful verification and policy evaluation

In the final scenario, the developer submits a pull request where when the verification service is run, it produces a report that is compliant with the policy, and OPA evaluates the report successfully. Resulting in the `verify_and_validate` job passing successfully, and the merge is allowed to proceed.

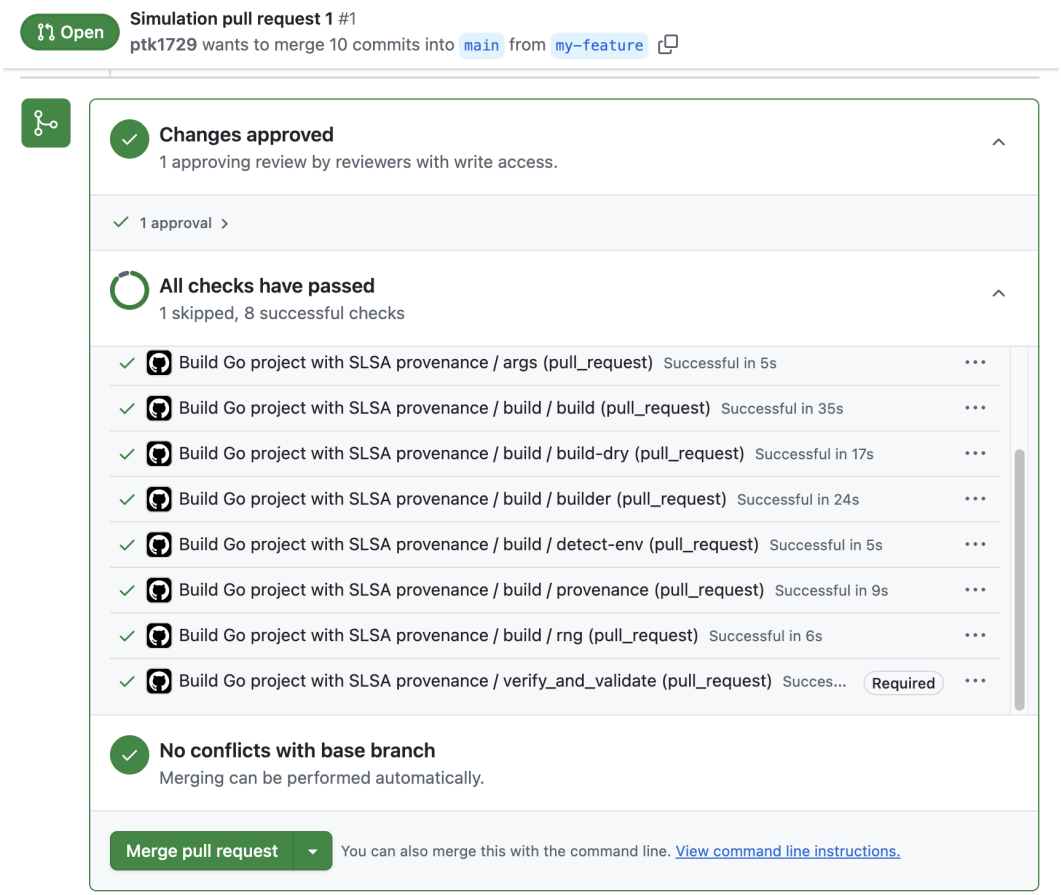

Figure 5.9: Successful run of the verification and policy evaluation workflow allowing the merge.

5.2.5 Discussion

The results confirm that policy-as-code can be effectively integrated into a standard GitHub development workflow. The simulation illustrates:

- Enforcement of verified commits and provenance-based validation.
- Prevention of unauthorized or non-compliant merges.
- Controlled override through administrative roles.

While the policy-as-code demonstration guarantees the functional correct behavior and integration, it is yet to be shown how secure the system is. In order to perform the evaluation, in the second part we analyze the verifier's attack surface applying a formal threat model.

5.3 Security Analysis: A Threat Model of the Verifier Service

A critical part of evaluating any security system is to apply a rigorous threat model to the system itself.

5.3.1 Introduction and Methodology

While the Verifier Service is intended to increase the security of a software supply chain, the Verifier Service opens up its own attack surface and potential vulnerabilities. In order to organize the identification and analysis of these risks, the formal threat model must be carried out.

For this analysis, we use the STRIDE framework, which is a well-established method for threat modeling created by Microsoft. STRIDE gives an acronym for six categories of threats: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. By focusing on the Verifier Service through each of these lenses, we can identify weaknesses in the Verifier Service's architecture, implementation and operational context.

5.3.2 Analysis of Threats

STRIDE-based analysis was used to identify and categorize threats to the Verifier Service. Each threat is evaluated in terms of its impact on system reliability and empirical likelihood based on the evaluation results.

Spoofing An attacker with repository write access could replace the legitimate `./verifier` binary in the workflow (e.g., `.github/workflows/main.yml`) with a malicious one that always produces passing results. Similarly, compromising a Governor's account would allow policy modification in `policy.rego` (e.g., disabling critical checks).

Impact: High, as both attacks fully bypass verification.
Likelihood: Medium, mitigated by GitHub's branch protection and CODEOWNERS but dependent on account hygiene and access control.
Effect: Undermines trust in automated attestation and policy enforcement.

Tampering Although reports are cryptographically hashed to prevent in-transit alteration, workflow tampering remains possible. An attacker could modify the OPA invocation (e.g., appending `true` or using `continue-on-error`) or manipulate external data from the Open Source Vulnerability (OSV) API through a man-in-the-middle response.

Impact: Medium-High, as policy validation could be silently bypassed.
Likelihood: Medium, given CI workflows are editable and external API traffic is trusted.
Effect: May cause false-positive compliance results or suppression of vulnerability reports.

Repudiation Without immutable logs or verifiable audit trails, actors could deny their actions or modify historical verification results.

Impact: Medium, since current logs depend on GitHub's retention policy.
Likelihood: Low-Medium, mitigated if workflow logs and reports are archived with signatures.
Effect: Reduces accountability in policy violations and compromises traceability of governance decisions.

Information Disclosure Verification reports expose detailed internal data such as vulnerabilities, open ports, and environment variables. If CI artifacts or logs are publicly visible, this information can assist adversaries in targeted attacks.

Impact: High (data leakage directly aids exploitation).
Likelihood: Medium, as CI logs are often world-readable in public repositories.
Effect: Empirically significant; sensitive metadata in JSON reports demands restricted artifact access and redaction mechanisms.

Denial of Service As shown in Table 5.1, verifying the `kubernetes` repository required approximately 1769 s (≈29 min). An attacker could exploit this by submitting pull requests with large commit histories or malformed configuration files, exhausting CI runner capacity.

Impact: High, potentially halting the CI pipeline for all users.
Likelihood: High, confirmed by empirical runtime measurements indicating poor scalability of the sequential design.
Effect: Demonstrates an operational vulnerability that must be mitigated through incremental or parallel verification.

Elevation of Privilege The verifier executes system utilities such as `git`, `gofmt`, and `golangci-lint`. Unsanitized inputs (e.g., filenames or commit messages containing shell metacharacters) may enable command injection on CI runners. Governor override privileges also represent a single point of trust.

Impact: Critical–could lead to remote code execution and repository compromise.
Likelihood: Medium, depending on input sanitization and runner isolation.
Effect: The most severe class of threat; calls for sandboxed execution and privilege separation between verification and policy evaluation.

Table 5.3: Summary of identified threats and mitigations (STRIDE framework).

Category	Example Threat	Impact	Mitigation
Spoofing	Malicious actor replaces verifier binary in CI workflow	False attestation and bypassed security gate	Signed workflow files and checksum validation of binaries
Tampering	Modification of OPA step (e.g., using 'continue-on-error')	Bypassed policy enforcement	Enforce immutable workflows and branch protection rules
Information Disclosure	Leakage of JSON reports with vulnerabilities	Exposure of internal topology and secrets	Restrict artifact visibility and redact sensitive fields
Denial of Service	Repository with excessive commit count	CI queue saturation and long delays	Introduce commit sampling or caching
Elevation of Privilege	Command injection in verifier	Remote code execution on runner	Input sanitization and least-privilege runner configuration

Table 5.3 summarizes the assessed risk levels and primary mitigations.

Together with the empirical evaluation, the threat model identifies architectural and operational hazards that have not been resolved. With this in mind, the following section begins with critical analysis of the conceptual and technical shortcomings of the current prototype.

5.4 Limitations and Critical Analysis

In addition to external threats, there is an internal analysis that involves a critical assessment of the system design decisions made, trade-offs, and conceptual constraints. This section discusses the paradoxes and weaknesses revealed during the course of implementation and evaluation of the proof-of-concept.

Determinism vs. Liveness Paradox One of the key designs objectives of this thesis was to have a deterministic, reproducible verification where a particular repository state will

produce the same report, which will be verifiable. However, on further scrutiny a fundamental tension between this objective and the nature of vulnerability scanning is exposed.

Deterministic checks such as formatting and linting only rely on the source code. Vulnerability scanning, in contrast, is a "liveness" check - its result depends on the state of an external and constantly changing vulnerability database. As a result, a build artifact previously declared to be secure and all tests ran on Monday may, without any alteration to their code or dependencies, be re-assessed to be insecure on Tuesday after a new CVE was announced.

This contradiction nullifies the promise of real reproducibility and leads to a challenging governance problem, namely what does the signature by a Governor on a release really mean when the security status on which it was based is not permanent and can be retroactively revoked. It implies that liveness-check based attestations cannot be made permanent statements of integrity.

Scalability and Architectural limitations. The proof-of-concept used today was built on a monolithic and sequential execution model and suffers from severe limitations both in terms of scalability and security.

Scalability: The verifier re-scans the whole repository on all executions, and verifies the commit history in its entirety. As the 29 minute long runtime for the Kubernetes repository showed, this architecture is not feasible for the large-scale monorepos that are popular in enterprise environments, where build checks on pull requests have to finish in minutes, if not seconds.

Security (Weak Trust Boundary): the design implements the measurement stage (verifier) and the adjudication stage (OPA evaluation) as part of a single CI job and execution environment. This results in a weak trust boundary. Because of a vulnerability in the verifier, which was one of the chain elements discovered in the Elevation of Privilege threat in Section 5.3.2, one command injection vulnerability can be used to damage the entire chain. An attacker would be able to abuse the verifier to create a falsified report in memory and pass it directly to the OPA engine for a successful evaluation. In this case there is no protection provided by the cryptographic hash, as the tampered report would have been hashed after the tampering took place and so the attack would not be visible to the policy engine.

Narrow Evaluation Scope The evaluation described in this thesis was purposefully limited to the Go ecosystem in order to provide a working proof-of-concept. On the one hand, this made it possible to integrate with the Go toolchain on a deep level; however, it also restricts the extrapolation of the results.

The suggestion that the system offers a "flexible and powerful foundation" for automated governance is rather an enlightened guess as much as a proven conclusion. The development of the system to other major programming ecosystems would give new and unique challenges that were not discussed in this work. To take the example of a Python implementation, it would have to grapple with complicated dependency management programs (pip vs. Poetry) and virtual environments.

Likewise, a Java integration would need an integration with build systems such as Maven or Gradle. Therefore, while the basic idea of separating the generation of evidence and policy

evaluation is sound, its implementation in a polyglot environment is an open question for further study.

Together, these results and analyses provide a comprehensive assessment of the system's performance, security posture, and design constraints. The next chapter concludes this thesis by summarizing the main findings and outlining directions for future work.

6 Conclusion and Further Work

This chapter concludes the thesis by summarizing its core contributions, evaluating the outcomes in relation to the initial problem statement, and proposing concrete directions for future research and enhancement.

6.1 Conclusion

The central argument of the thesis is that the demand for rapid delivery, which defines modern DevOps, fundamentally conflicts with the requirement to maintain strict security standards in the software supply chain. Manual review and human approval gate systems of governance have become a major bottleneck. These processes are not only slow but can also be prone to inconsistency and human error [81, 51, 72] thus posing security loopholes that can be exploited by malicious actors[40, 9], as exhibited by high profile supply-chain attacks. To address this challenge, an automated verification framework was researched, designed, and implemented to replace manual controls with deterministic, policy-based, and cryptographically verifiable mechanisms.

A proof-of-concept Verifier Service was developed to demonstrate the practicality of the proposed approach. The two-phase structure of the system is purposely designed to separate evidence-generation and policy adjudication, which forms a strong and reliable workflow.

Firstly, a complete scan of a source-code repository is performed, running an ensemble of modular tests. A cryptographic hash is used to provide basic protection over the integrity of this evidence. Secondly, an individual Open Policy Agent (OPA) extension is used to check the report against declarative security policies. The customized plugin of OPA is also one of the important contributions as it transforms OPA to house complex validation logic which may not be possible or preferred to be written in Rego. One of the critical findings made in this piece of work is the transformative efficiency and reliability of the automated method as opposed to that of the manual one. This Verifier Service produces a comprehensive report, with heterogeneous evidence items.

To a human reviewer, to carefully check this information with an intricate system of organizational policies would prove to be a tedious undertaking. That could take minutes to hours, of careful cross-checking, and would be a strain on the person, as well as would not be a foolproof method. The chance of a critical vulnerability being ignored or an unsigned

commit being missed is easy because of fatigue in the reviewer. Contrastingly, the OPA policy engine takes the whole structured report and compares all the rules to it within milliseconds.

This is a shift of subjective, and ad hoc human check to objective, machine-enforced governance where a certain degree of assurance is offered such that manual processes cannot possibly match. It was proved to be feasible through performance testing on repositories of all sizes, and it took between seconds and less than thirty minutes (extreme scenario) to execute programs based on a small project to a massive codebase like Kubernetes. The extra overhead was not significant when included in a real-world GitHub Actions pipeline and acceptable by the majority of development processes. The end-to-end demonstration proved the ability of the system to serve as an automated gatekeeper and prevent non-compliant pull requests, as well as giving easy-to-understand and actionable feedback. It serves as a blueprint of the organizations that wish to incorporate trust and security into their software delivery lifecycle.

While the prototype validates the core thesis assumptions, several open challenges and extensions remain, forming the basis of the following outlook.

6.2 Outlook and Further Work

This thesis demonstrated that verifiable, policy-based automation can reconcile the long-standing tension between development velocity and supply-chain security. The presented verifier prototype shows that automated evidence generation and policy enforcement are both feasible and effective. By reducing human error while preserving accountability, the approach offers a viable path toward continuous, auditable software assurance.

6.2.1 Expanding Ecosystem and Tooling Support

The current implementation is primarily focused on the Go ecosystem. The most immediate and valuable extension is to incorporate support for other major programming languages and their corresponding toolchains. This would involve developing new, modular verification plugins for:

Python : Integrating formatters like `black`, linters such as `flake8` or `pylint`, and dependency vulnerability scanners that work with `requirements.txt` or `poetry.lock` files.

Java : Adding support for the Maven or Gradle build systems, incorporating static analysis tools like `Checkstyle` or `SpotBugs`, and integrating with dependency checkers that analyze the project's POM or build scripts.

Rust : Including modules for the canonical `rustfmt` and the powerful `clippy` linter, along with tools like `cargo-audit` for vulnerability scanning.

Beyond application code, the analysis of infrastructure-as-code (IaC) can be significantly enhanced. The current manifest scanner for Dockerfiles and Kubernetes YAML could be extended to parse and validate Helm charts, Terraform plans, and Pulumi stacks. This would give deep security sensitivity insights, into the environment deployed in itself, to allow policies that prevent typical misconfigurations like publicly accessible storage buckets, overly generous IAM roles, or unencrypted secrets to be deployed before they are deployed.

6.2.2 Deepening Policy Enforcement and Verification Logic

The system's modular architecture is explicitly designed for extensibility. Further developments of the work should be aimed at introducing more complex and context-sensitive policy

checks. One example is test coverage analysis, in which policies can insist on a minimum line or branch coverage percentage (e.g. more than 80%) as a condition before merging code itself, thus providing a guarantee of some minimum quality and a smaller chance of regressions.

Moreover, the strength of the custom OPA Go-written plugin is hard to overestimate. It can be extended to do any custom check which is hard or impossible to write in Rego alone, as illustrated. This prepares the way to very customized organizational governance. For example, a custom check could be written to query an internal asset management database to ensure a project's dependencies are sourced only from approved vendors, or to cross-reference a feature flag system to verify that experimental code is not being enabled in a production build. This makes the system's logic infinitely extensible to meet specific business and security requirements.

6.2.3 Bridging Pre-Deployment and Runtime Security

This work has been deliberately scoped to pre-deployment verification, aligning with the 'shift-left' security paradigm. A powerful future direction would be to bridge this with runtime security monitoring, effectively closing the DevSecOps feedback loop. By integrating data from runtime tools like Falco or Cilium, the system could create adaptive policies. For instance, if a runtime security agent detects active exploitation of a specific CVE in production, that information could be fed back into the verifier's policy engine. The system could then automatically update its policy to block any new pull request that introduces or retains the vulnerable dependency, making the CI/CD pipeline responsive to real-world threats in near-real-time.

6.2.4 Addressing Implementation Challenges

As the system evolves, certain challenges will require dedicated research and engineering effort.

Performance at Scale : While the system handled the Kubernetes repository, the 29 minute runtime highlights the performance challenges of certain checks, particularly full commit history verification. Future work should investigate optimization strategies such as incremental analysis (only checking new commits in a pull request), intelligent caching of verification results, and parallelizing independent checks to reduce overall latency.

Maintaining Determinism : Ensuring that verification runs are perfectly reproducible is non-trivial. The versions of linters, scanners, and even the Go compiler can affect the output. Moreover, checks that rely on external data, like vulnerability databases, are inherently non-deterministic over time. A robust solution would involve using containerized, version-pinned toolchains for all checks and potentially snapshotting or proxying external data sources to ensure that a build from six months ago can be re-verified against the exact same context.

6.2.5 Solving the Determinism Problem in the Verifier Service

One of the identified key open challenges in this work is that vulnerability scanning phase is non-deterministic. Because the verifier makes an external vulnerability database call, the results are based upon the ever-changing state of that data source. A repository that is known to be 'secure' at a moment in time can be reported as later 'vulnerable' and have no

re-writes to the code or dependencies. This temporal discrepancy actually defeats the aims of reproducibility and auditability on which the verifier has its aspirations.

A good work-around would be to implement a local mirror or cached copy of the vulnerability database. Each verification execute would refer to a timestamped and versioned local dataset, and that all future re-verifications can then achieve the same results given the same context. Periodic or explicit administrative request would synchronize to the upstream database with no loss in determinism. This treatment makes verifier time constant and turns it into an accurate measurement device, supporting auditorable build trees over time and reproducible builds.

6.2.6 Alternative Approaches for Commit Signature Verification

The current commit verification mechanism will do a full history verification of all commits after a commit operation, which has a high guarantee of integrity but has a big runtime overhead, particularly in repositories which have long histories. Alternative methods can be investigated for the same assurance in a more efficient way.

One interesting solution is differential verification. Instead of validating all commits in the history, verifier could just validate commits that included in the current pull request or those that have not yet been verified and copied in a trusted cache. Also, the cache would document the cryptographic digests and signature meta-data of previously validated commits and future runs would elide redundant validation. Another method will be incremental verification based on Git principle of directed acyclic graph (DAG): New validator nodes are only required to have signatures validated for new nodes (commits) beyond a trusted basis. This would turn linear runtime to near constant norm for most development workflows while still providing the same level of assurance.

Such incremental verification strategies are compatible with practical continuous integration requirements and would guarantee a great improvement in scalability, without compromising the underlying security model.

6.2.7 Optimizations for Better Scalability

The evaluation results demonstrated that the verifier performs reliably but faces scalability limits when applied to large repositories such as `kubernetes`. Optimizing the verifier for speed and scalability is naturally a fairly valuable direction for further research.

The most direct improvement is file-level change detection. Instead of re-verifying the entire repository, the verifier should restrict analysis to files that have changed in a pull request. Combining this with dependency tracking would allow selective re-execution of only those checks affected by the modified components. Parallelization of independent verification stages and incremental caching of intermediate results would further reduce runtime without compromising coverage.

Additionally, it would be possible to have a distributed execution mode where various checks are delegated for an isolated runner or container that aggregate the results at the end of the pipeline Such architecture would exploit available CI/CD infrastructure parallelism, transforming the verifier from a sequential process into a horizontally scalable verification engine. With these optimizations, verification could remain within seconds even for large-scale industrial codebases, enabling adoption in performance-sensitive production pipelines. This would be a significant improvement in scalability and performance, and would make the verifier more useful in real-world production environments.

6.2.8 The Indispensable Role of Human Oversight

Lastly, it is important to realize the boundaries of automation. Despite extensive automation, human expertise remains essential in areas that require contextual understanding and creative reasoning:

Complex Architectural Reviews: The verifier service is capable of understanding whether a project is an approved authentication library but cannot determine whether the architecture of authentication and authorisation as a whole makes sense to the business logic it is protecting.

Detection of Logical Flaws: A policy can enforce that input is sanitized, but it cannot detect subtle logical bugs in business processes that could lead to financial or data integrity issues.

Novel Threat Modeling: Creative, context-aware threat modeling requires an understanding of an application's purpose, its users, and potential adversarial motives, a task far beyond the scope of automated policy.

This tool works as an effective, exhaustive support system that carries out the rest but not substitutes human governance. These identified directions collectively form a roadmap for evolving the verifier from a prototype into a comprehensive, industry-ready framework for automated software assurance.

6.2.9 Final Remarks

The journey toward a fully secure and automated software supply chain is ongoing, but this work marks a significant step forward. By successfully integrating evidence from supply chain modules and declarative policy enforcement, the implemented prototype moves beyond theoretical models and provides tangible confidence to a human in the loop.

It provides governors not with a replacement for their judgment, but with a powerful tool that replaces ambiguity with evidence and manual work with machine-enforced consistency. In doing so, it offers a compelling model for how organizations can speed up their security processes, rather than getting strangled by them. This work establishes that speed and integrity need not be competing priorities, but can work together.

Bibliography

[1] Paolo Missier, Khalid Belhajjame, and James Cheney. "The W3C PROV Family of Specifications for Modelling Provenance Metadata". In: *Proceedings of the 16th International Conference on Extending Database Technology (EDBT '13)*. ACM, 2013, pp. 773–776. URL: https://dl.acm.org/doi/10.1145/2452376.2452478.

[2] Luc Moreau et al. *PROV-DM: The PROV Data Model*. Tech. rep. Defines entities, activities, and agents, and relations such as wasGeneratedBy, used, and wasDerivedFrom. World Wide Web Consortium (W3C), 2013.

[3] ISACA. "Implementing Segregation of Duties: A Practical Experience Based on Best Practices". In: *ISACA Journal* (2016). URL: https://www.isaca.org/resources/isaca-journal/issues/2016/volume-3/implementing-segregation-of-duties-a-practical-experience-based-on-best-practices.

[4] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. *Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices*. arXiv preprint arXiv:1703.07019. Version accessed via https://arxiv.org/pdf/1703.07019. 2017.

[5] David Wilkinson et al. "Software Provenance: Track the Reality Not the Virtual". In: *Proceedings of the [Conference Name if known]*. ACM, 2018. DOI: 10.1145/3214239.3214244. URL: https://dl.acm.org/doi/10.1145/3214239.3214244.

[6] FIRST.Org, Inc. *Common Vulnerability Scoring System v3.1: Specification Document*. https://www.first.org/cvss/v3-1/specification-document. 2019.

[7] Santiago Torres-Arias et al. "in-toto: Providing farm-to-table guarantees for bits and bytes". In: *28th USENIX Security Symposium (USENIX Security '19)*. 2019, pp. 1393–1410. URL: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

[8] T. Bray. *The JSON Data Interchange Format: Canonicalization Scheme*. RFC 8785. 2020. DOI: 10.17487/RFC8785. URL: https://www.rfc-editor.org/rfc/rfc8785.

[9] M. Ohm et al. "A Review of Open Source Software Supply Chain Attacks". In: *Journal of Cybersecurity* 6.1 (2020), tzaa018. DOI: 10.1093/cybsec/tyaa018. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC7338168/.

[10] S. Rose et al. *NIST Special Publication 800-207: Zero Trust Architecture*. Tech. rep. National Institute of Standards and Technology, 2020. URL: https://doi.org/10.6028/NIST.SP.800-207.

[11] Matthew Zaber and Suku Nair. "A framework for automated evaluation of security metrics". In: *Proceedings of the 15th International Conference on Availability, Reliability and Security*. ARES '20. Virtual Event, Ireland: Association for Computing Machinery, 2020. ISBN: 9781450388337. DOI: 10.1145/3407023.3409197. URL: https://doi.org/10.1145/3407023.3409197.

[12] Inc. Codecov. *Post-Mortem / Root Cause Analysis: Codecov Bash Uploader Supply Chain Compromise*. online; April 15, 2021. https://about.codecov.io/apr-2021-post-mortem/. 2021.

[13] ISO/IEC JTC 1/SC 7. *ISO/IEC 5962:2021 - Software Package Data Exchange (SPDX)*. 2021. URL: https://www.iso.org/standard/81870.html.

[14] National Telecommunications and Information Administration (NTIA). *The Minimum Elements for a Software Bill of Materials (SBOM)*. 2021. URL: https://www.ntia.gov/files/ntia/publications/sbom_minimum_elements_report.pdf.

[15] Anqi Xu, Andrew Meneely, and Meiyappan Nagappan. "Why Security Defects Go Unnoticed during Code Reviews: A Case Study on Chromium OS". In: *arXiv preprint arXiv:2102.06909* (2021). URL: https://arxiv.org/abs/2102.06909.

[16] OWASP Foundation. *OWASP Top 10 CI/CD Security Risks*. Online at https://owasp.org/www-project-top-10-ci-cd-security-risks/. Accessed October 27, 2025. 2022.

[17] HashiCorp. *Using Open Policy Agent with Terraform*. 2022. URL: https://developer.hashicorp.com/terraform/tutorials/policy/opa.

[18] *ISO/IEC 27001:2022 Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements*. International Organization for Standardization, 2022. URL: https://www.iso.org/standard/82875.html.

[19] Avantika Mathur. *Introducing Harness Policy as Code, Powered by OPA*. https://www.harness.io/blog/harness-policy-as-code. Accessed: 2025-10-21. May 2022.

[20] Red Hat Engineering. *Implementing Policy-as-Code in CI/CD with Open Policy Agent*. 2022. URL: https://developers.redhat.com/articles/2022/05/04/policy-as-code-cicd-opa.

[21] Nuno Saavedra and João F. Ferreira. *GLITCH: Automated Polyglot Security Smell Detection in Infrastructure as Code*. 2022. arXiv: 2205.14371 [cs.CR]. URL: https://arxiv.org/abs/2205.14371.

[22] *SPDX® Specification Version 2.3*. https://spdx.github.io/spdx-spec/v2.3/. 2022.

[23] Bundesamt für Sicherheit in der Informationstechnik (BSI). *BSI IT-Grundschutz-Kompendium 2023*. 2023. URL: https://www.bsi.bund.de/IT-Grundschutz.

[24] Ramaswamy Chandramouli, Frederick Kautz, and Santiago Torres-Arias. *NIST Special Publication 800-204D: Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. Tech. rep. National Institute of Standards and Technology, 2023. URL: https://doi.org/10.6028/NIST.SP.800-204D.

[25] in-toto contributors. *in-toto Specification, Version 1.0*. GitHub repository at https://github.com/in-toto/specification/blob/v1.0/in-toto-spec.md. Version 1.0.0, June 02, 2023. 2023.

[26] Cybersecurity and Infrastructure Security Agency (CISA). *Applying Zero Trust Principles to CI/CD Environments*. 2023. URL: https://www.cisa.gov/resources-tools/resources/zero-trust-maturity-model.

[27] European Parliament and Council. *Directive (EU) 2022/2555 on Measures for a High Common Level of Cybersecurity across the Union (NIS2 Directive)*. Official Journal of the European Union, L 333/80. 2023. URL: https://eur-lex.europa.eu/eli/dir/2022/2555/oj.

[28] *Four Eyes Principle*. Open Risk Manual. 2023. URL: https://www.openriskmanual.org/wiki/Four_Eyes_Principle.

[29] D. Garcia, H. Lu, and M. Mirakhorli. "Policy Anchors for Secure Software Supply Chains: Interpreting NIST and CISA Guidance". In: *Journal of Software Supply Chain Security* (2023). URL: https://www.cisa.gov/sites/default/files/publications/ESF_SECURING_THE_SOFTWARE_SUPPLY_CHAIN_DEVELOPERS.PDF.

[30] Google. "BeyondProd: A Zero Trust Approach to CI/CD". In: (2023). URL: https://cloud.google.com/docs/security/beyondprod.

[31] Google Security Blog. *Policy-based Supply Chain Validation at Build Time*. 2023. URL: https://security.googleblog.com/2023/09/policy-based-supply-chain-validation.html.

[32] Antigoni Kruti, Usman Butt, and Rejwan Bin Sulaiman. *A review of SolarWinds attack on Orion platform using persistent threat agents and techniques for gaining unauthorized access*. arXiv preprint arXiv:2308.10294. cs.CE; cs.CR. 2023. DOI: 10.48550/arXiv.2308.10294.

[33] Eric Newcomer et al. *Static Analysis Results Interchange Format (SARIF) Version 2.1.0*. https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html. OASIS standard defining a JSON schema for static analysis tool results to improve interoperability. 2023.

[34] OASIS Open Project. *Static Analysis Results Interchange Format (SARIF) Version 2.1.0*. 2023. URL: https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html.

[35] Marc Ohm and Charlene Stuke. "SoK: Practical Detection of Software Supply Chain Attacks". In: *Proceedings of the 18th International Conference on Availability, Reliability and Security*. ARES '23. Benevento, Italy: Association for Computing Machinery, 2023. ISBN: 9798400707728. DOI: 10.1145/3600160.3600162. URL: https://doi.org/10.1145/3600160.3600162.

[36] Open Policy Agent Project. *Open Policy Agent Documentation*. 2023. URL: https://openpolicyagent.org/docs/latest/.

[37] Open Policy Agent Project. *Open Policy Agent: Policy-based Control for Kubernetes*. 2023. URL: https://www.openpolicyagent.org/docs/latest/kubernetes-introduction/.

[38] ReversingLabs. *5 CI/CD Breaches That Should Change How You Think About Software Supply Chain Security*. 2023. URL: https://www.reversinglabs.com/blog/cd-security-breaches-update-software-security-approach.

[39] *Securing the Software Supply Chain: Recommended Practices for Developers*. Tech. rep. Cybersecurity and Infrastructure Security Agency (CISA), 2023. URL: https://www.cisa.gov/sites/default/files/publications/ESF_SECURING_THE_SOFTWARE_SUPPLY_CHAIN_DEVELOPERS.PDF.

[40] Center for Security Studies (CSS). *Software Supply Chain Attacks: An Illustrated Typological Review*. Tech. rep. ETH Zurich, 2023. URL: https://css.ethz.ch/content/dam/ethz/special-interest/gess/cis/center-for-securities-studies/pdfs/Cyber-Reports-2023-01-Software-Supply-Chain-Attacks.pdf.

[41] SLSA Community. *Using OPA to Enforce SLSA Provenance Requirements*. 2023. URL: https://slsa.dev/blog/opa-integration.

[42] *Supply-chain Levels for Software Artifacts (SLSA) specification v1.0*. https://slsa.dev/spec/v1.0/. 2023.

[43] Wei Zhang and Young Kim. "SCM2: Reference Architecture for Software Supply Chain Metadata Management". In: *arXiv preprint arXiv:2310.06300* (2023). URL: https://arxiv.org/abs/2310.06300.

[44] Cloud Security Alliance. "Exploring DevSecOps Tools and Best Practices". In: *Cloud Security Alliance Article Series* (2024). URL: https://cloudsecurityalliance.org/articles/devsecops-tools.

[45] Cybersecurity and Infrastructure Security Agency (CISA). *Secure Software Development Attestation Form*. 2024. URL: https://www.cisa.gov/secure-software-attestation-form.

[46] *CycloneDX Specification Overview*. https://cyclonedx.org/specification/overview/. 2024.

[47] European Parliament and Council. *Regulation (EU) 2024/2847 on Horizontal Cybersecurity Requirements for Products with Digital Elements (Cyber Resilience Act)*. Official Journal of the European Union, L 2024/2847. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[48] Finite State. *Software Supply Chain Security Assessment Framework*. Finite State Blog. 2024. URL: https://finitestate.io/blog/software-supply-chain-security-assessment-framework.

[49] GitHub, Inc. *GitHub Code Scanning and SARIF Support*. https://docs.github.com/en/code-security/code-scanning/integrating-with-code-scanning/uploading-a-sarif-file-to-github. Documents GitHub's adoption of SARIF for static analysis result ingestion. 2024.

[50] Tobias Hartmann and Robert Müller. "Policy-as-Code: Enforcing Governance with Open Policy Agent (OPA)". In: *ResearchGate Preprint* (2024). URL: https://www.researchgate.net/publication/391016222_Policy-as-Code_Enforcing_Governance_with_Open_Policy_Agent_OPA.

[51] Eman Abu Ishgair, Marcela S. Melara, and Santiago Torres-Arias. *SoK: A Defense-Oriented Evaluation of Software Supply Chain Security*. arXiv preprint arXiv:2405.14993. 2024. URL: https://arxiv.org/abs/2405.14993.

[52] Emily Jack, Harold Castro, and Shreyan Sarkar. "Policy-as-Code: Enforcing Governance with Open Policy Agent (OPA)". In: *arXiv preprint* (2024). Explores the use of OPA for policy as code across cloud-native environments. URL: https://www.researchgate.net/publication/391016222%5C_Policy-as-Code%5C_%20Enforcing%5C_Governance%5C_with%5C_Open%5C_Policy%5C_Agent%5C_OPA.

[53] Theo Jansen and Martin Krause. "Automated Compliance Checks with Open Policy Agent (OPA) in Multi-Cloud". In: *ResearchGate Preprint* (2024). URL: https://www.researchgate.net/publication/392163378_Automated_Compliance_Checks_with_Open_Policy_Agent_OPA_in_Multi-Cloud.

[54] Yizheng Liu, Sascha Fahl, Miryung Kim, et al. "Dirty-Waters: Detecting Software Supply Chain Smells". In: *arXiv preprint arXiv:2410.16049* (2024). Available at https://arxiv.org/abs/2410.16049.

[55] Mehdi Mirakhorli et al. "A Landscape Study of Open Source and Proprietary Tools for Software Bill of Materials (SBOM)". In: *arXiv preprint arXiv:2402.11151* (2024). URL: https://arxiv.org/abs/2402.11151.

[56] OWASP Foundation. *DefectDojo: Application Security Orchestration and Vulnerability Management*. https://docs.defectdojo.com/. Describes DefectDojo's integration, deduplication, and dashboard capabilities for security findings. 2024.

[57] OWASP Foundation. *OWASP Top 10 CI/CD Security Risks*. 2024. URL: https://owasp.org/www-project-top-10-ci-cd-security-risks/.

[58] *Software Supply Chain Security in DevSecOps CI/CD Pipelines (SP 800-204D)*. Tech. rep. NIST, 2024. URL: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf.

[59] National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. Tech. rep. NIST SP 800-218 Rev.1. NIST, 2024. URL: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf.

[60] Yuchen Zhao, Haoyu Zhang, et al. "SoK: Analysis of Software Supply Chain Security". In: *arXiv preprint arXiv:2406.10109* (2024). URL: https://arxiv.org/abs/2406.10109.

[61] Thomas Zimmermann et al. "SoK: A Defense-Oriented Evaluation of Software Supply Chain Security". In: *arXiv preprint arXiv:2405.14993* (2024). URL: https://arxiv.org/abs/2405.14993.

[62] GitHub Documentation Contributors. *About GitHub Advanced Security*. Online at https://docs.github.com/en/get-started/learning-about-github/about-github-advanced-security. Accessed October 27, 2025. 2025.

[63] Open Policy Agent Contributors. *CI/CD — Open Policy Agent Documentation*. Online at https://www.openpolicyagent.org/docs/cicd. Accessed October 27, 2025. 2025.

[64] secure-systems-lab contributors. *DSSE: Dead Simple Signing Envelope Specification*. GitHub repository at https://github.com/secure-systems-lab/dsse. Last accessed October 27, 2025. 2025.

[65] Sigstore Contributors. *Overview of Sigstore*. Online at https://docs.sigstore.dev/about/overview/. Accessed October 27, 2025. 2025.

[66] Sowmiya Dhandapani. *Enhancing Software Supply Chain Security Through STRIDE-Based Threat Modelling of CI/CD Pipelines*. 2025. arXiv: 2506.06478 [cs.SE]. URL: https://arxiv.org/abs/2506.06478.

[67] *Forked Caddy Repository*. 2025. URL: https://github.com/ptk1729/caddy.

[68] *Forked Kubernetes Repository*. 2025. URL: https://github.com/ptk1729/kubernetes.

[69] *Go Project Repository*. 2025. URL: https://github.com/prateekanne/go_proj.

[70] Daniel Hugenroth et al. "Attestable Builds: compiling verifiable binaries on untrusted systems using trusted execution environments". In: *arXiv preprint arXiv:2505.02521* (2025). URL: https://arxiv.org/abs/2505.02521.

[71] Open Container Initiative. *Open Container Initiative (OCI)*. Online at https://opencontainers.org/. Defines open standards for container formats and runtimes. Accessed October 27, 2025. 2025.

[72] JFrog Ltd. *Software Supply Chain State of the Union 2025*. Tech. rep. JFrog, 2025. URL: https://s201.q4cdn.com/814780939/files/doc_presentations/2025/Apr/01/JFrog-Software-Supply-Chain-Report-2025.pdf.

[73] *OPA Plugin Repository*. 2025. URL: https://github.com/ptk1729/opa_plugin.

[74] Open Source Security Foundation. *Secure Supply Chain Consumption Framework (S2C2F)*. Tech. rep. Linux Foundation, 2025. URL: https://github.com/ossf/s2c2f/.

[75] OpenSSF. *A Maintainer's Guide: Securing CI/CD Pipelines After the tj-actions and Reviewdog Attacks*. 2025. URL: https://openssf.org/blog/2025/06/11/maintainers-guide-securing-ci-cd-pipelines-after-the-tj-actions-and-reviewdog-supply-chain-attacks/.

[76] OWASP Foundation. *OWASP DefectDojo*. Accessed: 2025-10-27. 2025. URL: https://owasp.org/www-project-defectdojo/.

[77] L. Prates and R. Pereira. "DevSecOps Practices and Tools: A Systematic Literature Review". In: *International Journal of Information Security* (2025). DOI: 10.1007/s10207-024-00914-z. URL: https://link.springer.com/article/10.1007/s10207-024-00914-z.

[78] *Supply-chain Levels for Software Artifacts (SLSA)*. https://slsa.dev/. Accessed: 2025-10-21. 2025.

[79] *Verification Service Repository*. 2025. URL: https://github.com/ptk1729/verifier_service.

[80] Laurie Williams et al. "Research Directions in Software Supply Chain Security". In: *ACM Trans. Softw. Eng. Methodol.* 34.5 (May 2025). ISSN: 1049-331X. DOI: 10.1145/3714464. URL: https://doi.org/10.1145/3714464.

[81] Abhijeet Ghadge et al. "Managing Cyber Risk in Supply Chains: A Review and Research Agenda". In: *Supply Chain Management: An International Journal* (). DOI: 10.1108/SCM-10-2018-0373. URL: https://www.researchgate.net/publication/335018536_Managing_Cyber_Risk_in_Supply_Chains_A_Review_and_Research_Agenda.

A Appendix

This appendix contains examples of the JSON and OPA artifacts used by the verification service.

A.1 Example JSON report schema with measurement results and attestation data

Listing A.1: Example JSON report schema with measurement results and attestation data


```
{
    "metadata": {
      "project_name": "Go test server",
      "repo_url": "https://github.com/ptk1729/go_proj",
      "commit_hash": "599ac3c717757fe7434a8e41f618fb0071cd8d95",
      "commit_message": "add docker and k8s manifest",
      "checked_at": "2025-10-11T00:31:54Z",
      "verifier_version": "1.0.0",
      "run_id": "3bbe171a-1da6-478a-ba75-4a977472f7c1",
      "verification_status": "FAILED",
      "report_sha256": "5a343f052cd8501a59f1cdf364598e347ab184c34061164a24e7d59627a73283"
    },
    "report": {
      "linting": {
        "status": "WARNING",
        "errors": null,
        "warnings": [
          "main.go:33:6: func ‘dummyDB’ is unused (unused)",
          "func dummyDB() {",
          "^",
          "main.go:43:6: func ‘dummyJWT’ is unused (unused)",
          "func dummyJWT() {",
          "^"
        ],
        "tool": "golangci-lint"
      },
      "formatting": {
        "status": "PASSED",
        "tool": "gofmt",
        "files_changed": null
      },
      "vulnerability_check": {
        "status": "FAILED",
        "tool": "osv-scanner",
        "vulnerabilities": [
          {
```

```
                "id": "\ac{GO}-2025-3849",
                "package": "stdlib",
                "ecosystem": "Go",
                "summary": "Incorrect results returned from Rows.Scan in database/sql",
                "details": "Cancelling a query (e.g. by cancelling the context passed to one of the
                    query methods) during a call to the Scan method of the returned Rows can result
                    in unexpected results if other queries are being made in parallel. This can
                    result in a race condition that may overwrite the expected results with those of
                    another query, causing the call to Scan to return either unexpected results from
                    the other query or an error.",
                "severity_object": {
                  "level": "",
                  "type": "",
                  "cvss_vector": "",
                  "score": ""
                },
                "fixed_version": "1.24.6"
              },
              {
                "id": "GO-2025-3956",
                "package": "stdlib",
                "ecosystem": "Go",
                "summary": "Unexpected paths returned from LookPath in os/exec",
                "details": "If the PATH environment variable contains paths which are executables (
                    rather than just directories), passing certain strings to LookPath (\"\", \".\",
                    and \"..\"), can result in the binaries listed in the PATH being unexpectedly
                    returned.",
                "severity_object": {
                  "level": "",
                  "type": "",
                  "cvss_vector": "",
                  "score": ""
                },
                "fixed_version": "1.24.6"
              }
            ]
          },
          "commit_verification": {
            "status": "FAILED",
            "no_verified_commits": 36,
            "no_unverified_commits": 3
          },
          "env_variables_check": {
            "status": "PASSED",
            "issues": null
          },
          "custom_checks": [
            {
              "name": "Dockerfile Best Practices",
              "status": "PASSED",
              "details": []
            }
          ],
          "slsa_check": {
            "status": "PASSED",
            "binary_path": "/home/prateek/go_proj/binary-linux-amd64/binary-linux-amd64",
            "provenance_path": "/home/prateek/go_proj/binary-linux-amd64.intoto.jsonl/binary-linux-
                amd64.intoto.jsonl",
            "source_uri": "git+https://github.com/ptk1729/go_proj",
            "slsa_level": "Unknown",
            "missing_provenance": false,
            "builder_id": "https://github.com/slsa-framework/slsa-github-generator/.github/workflows/
                builder_go_slsa3.yml@refs/tags/v2.1.0",
            "verified_requirements": [
              {
                "name": "binary_exists",
                "status": "PASSED"
```

```
94              },
95              {
96                "name": "provenance_exists",
97                "status": "PASSED"
98              },
99              {
100               "name": "digest_computed",
101               "status": "PASSED"
102             },
103             {
104               "name": "provenance_read",
105               "status": "PASSED"
106             },
107             {
108               "name": "signature_and_policy",
109               "status": "PASSED"
110             },
111             {
112               "name": "source_uri_match",
113               "status": "PASSED"
114             }
115           ]
116         },
117         "manifest_scan": {
118           "status": "PASSED",
119           "tool": "manifest-scanner",
120           "dockerfiles": [
121             {
122               "path": "repo_clone/Dockerfile",
123               "exposed_ports": [
124                 {
125                   "port": 8080,
126                   "protocol": "tcp",
127                   "type": "exposed"
128                 },
129                 {
130                   "port": 9090,
131                   "protocol": "tcp",
132                   "type": "exposed"
133                 }
134               ],
135               "environment_variables": [
136                 {
137                   "name": "APP_ENV",
138                   "value": "production \\",
139                   "type": "env"
140                 }
141               ],
142               "has_issues": false
143             }
144           ],
145           "kubernetes_manifests": [
146             {
147               "path": "repo_clone/test_deployment.yml",
148               "kind": "Deployment",
149               "name": "random-server",
150               "exposed_ports": [
151                 {
152                   "port": 8080,
153                   "protocol": "tcp",
154                   "type": "container"
155                 },
156                 {
157                   "port": 9090,
158                   "protocol": "tcp",
159                   "type": "container"
160                 }
```

```
                ],
                "environment_variables": [
                  {
                    "name": "ENV_MODE",
                    "value": "production",
                    "type": "env"
                  },
                  {
                    "name": "API_KEY",
                    "value": "",
                    "type": "env"
                  }
                ],
                "has_issues": false
              }
            ],
            "total_files": 2,
            "files_with_issues": 0
          }
        }
      }
```

A.2 OPA policy file

Listing A.2: Example OPA policy file

```
package report.verify

import rego.v1

  # --- config -----------------------------------------------------
  default required := {
    "linting":                true,
    "formatting":             true,
    "vulnerability_check":    true,
    "commit_verification":    true,
    "env_variables_check":    false,
    "slsa_check":             true
  }

  default warn_as_fail := {
    "linting": false,
  }

  allowed_ports := {80, 443, 8080, 9090}
  allowed_env_vars := {"APP_ENV", "LOG_LEVEL", "DB_HOST", "DB_USER", "API_KEY", "NAME", "SECRET"}

  # ----------------------------------------------------------------

  flat_report := inspect.print_report_strings(input)

  default stored_hash := ""
  stored_hash := s if {
    input.metadata
    s := input.metadata.report_sha256
    s != null
    s != ""
  }

  missing_stored if stored_hash == ""

  calculated_hash := crypto.sha256(flat_report)

  hash_match if {
  print("calculated_hash")
```

```
print(calculated_hash)
  not missing_stored
  stored_hash == calculated_hash
}

# helpers
is_required(name) := b if {
  b := required[name]
} else := false

status_bad(name, s) if s == "FAILED"
status_bad(name, s) if {
  warn_as_fail[name]
  s == "WARNING"
}

section_failures := {
  k: sect |
  some k
  sect := input.report[k]
  is_required(k)
  status_bad(k, sect.status)
}

custom_failures := {
  sprintf("custom:%s", [item.name]): item |
  some i
  item := input.report.custom_checks[i]
  n := sprintf("custom:%s", [item.name])
  is_required(n)
  status_bad(n, item.status)
}

failures := object.union(section_failures, custom_failures)

deny contains msg if {
  missing_stored
  msg := "metadata.report_sha256_is_missing"
}

deny contains msg if {
  not missing_stored
  calculated_hash != stored_hash
  msg := sprintf("hash_mismatch:_stored=%s_calculated=%s", [stored_hash, calculated_hash])
}

deny contains msg if {
    manifest := input.report.manifest_scan.kubernetes_manifests[_]
    port_info := manifest.exposed_ports[_]
    print("port_info", port_info)
    not port_info.port in allowed_ports
    msg := sprintf("Port_%v_in_%v_is_not_allowed", [port_info.port, manifest.name])
}

deny contains msg if {
  some k
  failures[k]
  msg := sprintf("required_check_failed:_%s", [k])
}

deny contains msg if {
    manifest := input.report.manifest_scan.kubernetes_manifests[_]
    port_info := manifest.exposed_ports[_]
    not port_info.port in allowed_ports
    msg := sprintf("Port_%v_in_%v_is_not_allowed", [port_info.port, manifest.name])
}
```

```
summary := {
  "hash": {
    "stored":     stored_hash,
    # "calculated": calculated_hash,
    # "match":      hash_match,
  },
  "required":     required,
  "warn_as_fail": warn_as_fail,
  "deny":         deny,
  "ok":           count(deny) == 0,
}

result := {
  "summary":  summary,
  "failures": failures,
}
```

A.3 OPA plugin output

Listing A.3: Example OPA plugin output

```
{
  "failures": {},
  "summary": {
    "deny": [],
    "hash": {
      "calculated": "0ce85ae7167405029d9a9a0edaca938fe7b8f390926aa83f3ff360c05d6801f9",
      "match": true,
      "stored": "0ce85ae7167405029d9a9a0edaca938fe7b8f390926aa83f3ff360c05d6801f9"
    },
    "ok": true,
    "required": {
      "commit_verification": false,
      "custom:Dockerfile Best Practices": false,
      "env_variables_check": true,
      "formatting": true,
      "linting": true,
      "slsa_check": true,
      "vulnerability_check": false
    },
    "warn_as_fail": {
      "linting": false
    }
  }
}
```

A.4 OPA evaluation failure due to a port not allowed

Listing A.4: Example of a OPA evaluation failure due to a port not allowed

```
  {
    "failures": {},
    "summary": {
      "deny": [
        "Port 9090 in random-server is not allowed" @@
      ],
      "hash": {
        "stored": "fccf55a1c30e51f4f1ac26b9bc8960f8388457c68cb97203df023ea5b9266abd"
      },
      "ok": false,
      "required": {
        "commit_verification": false,
        "env_variables_check": false,
```

```
14          "formatting": true,
15          "linting": true,
16          "slsa_check": true,
17          "vulnerability_check": false
18        },
19        "warn_as_fail": {
20          "linting": false
21        }
22      }
23    }
```